



UNIVERSITÉ DE  
MONTPELLIER



STATISTIQUE  
SCIENCE DES DONNÉES BIOSTATS  
UNIVERSITÉ DE MONTPELLIER



FACULTÉ DES SCIENCES  
DE MONTPELLIER

# M2 STATISTIQUES ET SCIENCES DES DONNÉES – BIOSTATISTIQUES –

## RAPPORT DE STAGE

---

### Titre du stage

---

Guillaume BOULAND

*Tuteur académique :*

Nicolas Meyer

*Tuteurs professionnels :*

Mai Nguyen-Chi

Benjamin Charlier

*Jury :*

Ali Gannoun

Élodie Brunel

Nicolas Meyer



2023 – 2024

Résumé

Résumé

## Abstract

# Remerciements

blabla

# Table des matières

|                                                                                       |           |
|---------------------------------------------------------------------------------------|-----------|
| <b>Introduction</b>                                                                   | <b>8</b>  |
| Cadre du stage . . . . .                                                              | 8         |
| Contexte et enjeux . . . . .                                                          | 8         |
| Objectifs du stage . . . . .                                                          | 9         |
| Plan du rapport . . . . .                                                             | 10        |
| <b>1 Contexte biologique</b>                                                          | <b>11</b> |
| 1.1 Macrophages, cellules immunitaires dynamiques . . . . .                           | 11        |
| 1.2 Signal calcique, candidat prometteur . . . . .                                    | 11        |
| 1.3 Zebrafish, modèle animal idoine . . . . .                                         | 11        |
| 1.4 Microscopie confocale . . . . .                                                   | 11        |
| <b>2 Segmentation automatique d’images microscopiques de macrophages</b>              | <b>12</b> |
| 2.1 Introduction à la segmentation d’images . . . . .                                 | 12        |
| 2.1.1 Image numérique . . . . .                                                       | 12        |
| 2.1.2 Différentes approches de segmentation d’images . . . . .                        | 13        |
| 2.1.3 Évaluation des performances . . . . .                                           | 14        |
| 2.2 Calibration de modèles de segmentation avec Mactrack2 . . . . .                   | 17        |
| 2.2.1 Cartesian Genetic Programming (CGP) . . . . .                                   | 17        |
| 2.2.2 Kartezio . . . . .                                                              | 19        |
| 2.2.3 En pratique . . . . .                                                           | 21        |
| 2.3 Résultats . . . . .                                                               | 26        |
| 2.3.1 Visualisation de la qualité de segmentation . . . . .                           | 26        |
| 2.3.2 Segmentation automatique d’images avec MacTrack2 . . . . .                      | 26        |
| 2.3.3 Performance/Résultat Comparaison des modèles selon le format d’images . . . . . | 27        |
| <b>3 Suivi temporel des macrophages</b>                                               | <b>29</b> |
| 3.1 Tracking à partir de la segmentation d’images . . . . .                           | 29        |
| 3.2 Post-traitement . . . . .                                                         | 33        |
| <b>4 Étude et reparamétrisation d’un modèle de fondation</b>                          | <b>36</b> |
| 4.1 Introduction . . . . .                                                            | 36        |
| 4.2 Fine-tuning de SAM-2 . . . . .                                                    | 38        |
| 4.2.1 Définition . . . . .                                                            | 38        |
| 4.2.2 Jeu de données . . . . .                                                        | 38        |

|          |                                                                                |           |
|----------|--------------------------------------------------------------------------------|-----------|
| 4.2.3    | <i>hyperparamètres</i>                                                         | 39        |
| 4.2.4    | <i>Fonctions de perte</i>                                                      | 39        |
| 4.2.5    | <i>Optimisation</i>                                                            | 41        |
| 4.2.6    | <i>Résultats et comparaisons</i>                                               | 41        |
| 4.3      | Propagation dans une vidéo                                                     | 41        |
| <b>5</b> | <b>Analyse des signaux calciques issues de la segmentation des macrophages</b> | <b>42</b> |
| <b>6</b> | <b>Conclusion et perspectives</b>                                              | <b>43</b> |
| 6.1      | Disponibilité du code                                                          | 43        |
| 6.2      | Liens utiles                                                                   | 43        |
| <b>7</b> | <b>Matériel et Méthodes</b>                                                    | <b>44</b> |
|          | Acquisition des images                                                         | 44        |
|          | Format des vidéos                                                              | 44        |
|          | Entraînement des modèles Kartezio                                              | 44        |
|          | Fine-tuning de SAM-2                                                           | 44        |
|          | <b>Bibliographie</b>                                                           | <b>47</b> |
| <b>A</b> | <b>Annexes</b>                                                                 | <b>48</b> |
| A.1      | Superposition des deux canaux                                                  | 48        |
| A.2      | Liste des fonctions de traitement d'images disponibles dans Kartezio           | 49        |
| A.3      | Tracking des macrophages par MacTrack                                          | 50        |

# Table des figures

|      |                                                                                                                                                                                                           |    |
|------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 2.1  | Images en 2-dimension de la nageoire blessée d'une larve de zebrafish réalisée par microscopie confocale. . . . .                                                                                         | 13 |
| 2.2  | Schématisation de l'IoU (figure issue de [10]). . . . .                                                                                                                                                   | 15 |
| 2.3  | Comparaison des annotations de deux opérateurs sur une image de macrophages. . . . .                                                                                                                      | 16 |
| 2.4  | . . . . .                                                                                                                                                                                                 | 18 |
| 2.5  | Exemple de graphe orienté acyclique de notre propre modèle (voir section 2.2). . . . .                                                                                                                    | 19 |
| 2.6  | Architecture d'un modèle généré par Kartezio. Figure issue de [3]. .                                                                                                                                      | 20 |
| 2.7  | Image annotée issue du jeu de donnée d'entraînement créé. . . . .                                                                                                                                         | 22 |
| 2.8  | Macrophages observés à différents temps post-amputation pour un même poisson. . . . .                                                                                                                     | 23 |
| 2.9  | Exemples de comparaisons entre la vérité annotée (en bleu) et les prédictions du modèle choisi (en rouge) sur des images issues de l'échantillon d'entraînement (à gauche) et de test (à droite). . . . . | 26 |
| 2.10 | Illustrations des différentes sorties de la fonction <code>locate</code> . . . . .                                                                                                                        | 27 |
| 3.1  | Exemple d'application de la fonction <code>defuse</code> . . . . .                                                                                                                                        | 31 |
| 3.2  | Résultats du tracking des macrophages sur une image de chaque canal de la vidéo. . . . .                                                                                                                  | 32 |
| 3.3  | Illustration du premier cas de fusion manuelle des macrophages. . .                                                                                                                                       | 33 |
| 3.4  | Illustration du second cas de fusion manuelle des macrophages. . . .                                                                                                                                      | 34 |
| 3.5  | . . . . .                                                                                                                                                                                                 | 35 |
| 4.1  | Architecture de SAM-2 ( <i>figure issue de [30]</i> ). . . . .                                                                                                                                            | 37 |
| 4.2  | Image d'origine (à gauche) comprenant des macrophages et leur détourage (à droite). . . . .                                                                                                               | 38 |
| A.1  | Superposition de deux canaux de couleurs, macrophages en rouge et calcium en vert. . . . .                                                                                                                | 48 |
| A.2  | Exemple d'application de la fonction <code>defuse</code> . . . . .                                                                                                                                        | 52 |
| A.3  | Résultats du tracking des macrophages sur une image de chaque canal de la vidéo. . . . .                                                                                                                  | 54 |

# Liste des tableaux

|     |                                                                                                                                                                                                                                                                                      |    |
|-----|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 2.1 | Extrait du fichier <code>dataset.csv</code> nécessaire pour la reconnaissance des images ( <i>input</i> ) et des annotations ( <i>label</i> ) pour Kartezio, selon leur appartenance ( <i>set</i> ) au jeu d'entraînement ( <i>training</i> ) ou de test ( <i>testing</i> ). . . . . | 24 |
| 2.2 | Exemples de performances des modèles entraînés sur les différents jeux de données. . . . .                                                                                                                                                                                           | 28 |
| 4.1 | Différents exemples des valeurs que peuvent prendre les sorties du <i>mask decoder</i> (logits), leur interprétation et la valeur de la probabilité associée (par transformation sigmoïde). . . . .                                                                                  | 41 |
| A.1 | Fonctions et descriptions . . . . .                                                                                                                                                                                                                                                  | 50 |



# Introduction

## Cadre du stage

L'ensemble des travaux de ce stage a été réalisé au sein du LPHI (*Laboratory of Pathogen Host Immunity*), un laboratoire mixte du CNRS (Centre National de la Recherche Scientifique) et de l'Université de Montpellier, situé sur le campus Triolet. Ce stage a été réalisé sous la supervision de Mai Nguyen-Chi, chercheuse CNRS au LPHI, et Benjamin Charlier, chercheur à l'INRAE (Institut National de Recherche pour l'Agriculture, l'Alimentation et l'Environnement) à Toulouse.

Les différentes équipes du LPHI, au nombre de 8 pour une quarantaine de chercheurs permanents, ont pour but d'apporter de nouvelles connaissances sur les interactions entre les pathogènes et l'hôte, notamment dans le cadre d'infections parasitaires, bactériennes et virales. La plateforme de microscopie MRI (Montpellier Ressources Imagerie), partenaire du LPHI, a été sollicitée pour l'acquisition des images de microscopie confocale utilisées dans ce stage.

L'équipe que j'ai intégrée, dirigée par Mai Nguyen-Chi et Laure Yatime, se concentre sur les mécanismes d'inflammations physiologique (blessure) et pathologique (infection), en particulier sur les macrophages, des cellules immunitaires jouant un rôle clé dans la réponse immunitaire. Elle est composée de trois groupes de chercheurs avec les deux premiers respectivement dirigés par Mai Nguyen-Chi et Laure Yatime, et le troisième par Étienne Lelièvre.

J'ai travaillé en collaboration étroite avec une étudiante en M2 Biologie Santé - Infection Biology, Norma Veltz, qui a aussi réalisé son stage de M2 au sein du LPHI sur la même période que le mien. Son travail, centré sur l'imagerie intravitale des signaux calciques ( $\text{Ca}^{2+}$ ) pour étudier les fonctions des macrophages (cellules immunitaires) lors d'inflammations stériles (blessure) et pathologiques (infection), a fourni les images nécessaires à mon stage. Ainsi, nos stages sont complémentaires : elle a réalisé les expériences biologiques et acquis les données microscopiques, tandis que mon travail a consisté à traiter et interpréter ces images.

## Contexte et enjeux

Les macrophages sont des cellules immunitaires polyvalentes. Ils jouent un rôle essentiel dans la réponse immunitaire en phagocytant (éliminant) les agents patho-

gènes et en régulant l'inflammation. Dotés d'une plasticité fonctionnelle, ils peuvent changer d'état en passant d'un état pro-inflammatoire (M1) essentiel pour la défense de l'hôte à un état anti-inflammatoire (M2) servant à la réparation tissulaire. Nous appelons ce processus de changement d'état fonctionnel la polarisation.

Des études récentes [1] ont montré que les signaux de calcium intracellulaires auraient un rôle dans l'activation des macrophages M1. Cependant, le lien entre les signaux calciques et la polarisation reste flou. En effet, nous ne connaissons pas les schémas de signaux calciques entre les différents états fonctionnels des macrophages. L'étude de ces signaux calciques semble être une voie prometteuse et pourrait permettre d'identifier ces états.

À l'aide d'un modèle animal de renom, le zebrafish, nous cherchons donc à définir des métriques pertinentes pour l'étude des signaux calciques dans les macrophages. Nous observons au microscope confocal des poissons transgéniques, ce qui nous permet de visualiser les macrophages et la présence de calcium dans ces cellules, en ayant un suivi longitudinal après une inflammation.

## Objectifs du stage

Afin de mieux comprendre la dynamique des signaux calciques, qui pourrait aider à identifier ces différents états, nous cherchons donc à développer un outil permettant d'identifier (segmenter), et suivre automatiquement chaque macrophage individuellement dans des images et vidéos de microscopie et mesurer l'activité calcique dans ces cellules afin d'établir la signature de ce signal dans les différents macrophages.

Tout d'abord, nous commencerons par réaliser la segmentation automatique d'une image réalisée par microscopie en utilisant *Kartezio*, un algorithme basé sur la programmation génétique cartésienne permettant de générer des pipelines de segmentation d'images efficaces et interprétables.

La librairie Python MacTrack2, qui instancie cette méthode, aura pour but d'être intuitive, car destinée à être utilisée par des biologistes. Elle contiendra des scripts exécutables en un clic et avec une documentation détaillée disponible sur un site internet (voir **Disponibilité du code**).

Un film (ou vidéo) est un enchaînement d'images. Chaque image étant déjà segmentée, nous étendrons cette segmentation au domaine temporel, en réalisant un tracking des macrophages dans ces vidéos. Pour ce faire, nous utiliserons un modèle de fondation, SAM-2, que nous affinerons à l'aide d'un *fine-tuning* sur nos données vidéos. Nous espérons ainsi améliorer le suivi (tracking) des macrophages en segmentant directement la vidéo entière, plutôt que de traiter les images une par une.

L'objectif final de ce travail est donc de proposer une solution complète, auto-

matisée et adaptée aux besoins des biologistes pour étudier la signature calcique des macrophages à travers l'espace et le temps.

## Plan du rapport

Le plan du rapport suivra la description des objectifs présentés ci-dessus. Le premier chapitre portera sur le contexte biologique de ce stage afin de décrire les enjeux de la polarisation de macrophages, l'importance des signaux calciques et les méthodes d'acquisition des images de microscopie.

Le second chapitre comprendra une introduction sur la segmentation d'images et présentera la méthode choisie permettant de segmenter automatiquement des objets dans des images, suivi des résultats obtenus sur nos données.

Nous aborderons ensuite l'extension de cette segmentation au domaine temporel, en présentant d'abord les améliorations apportées à la librairie MacTrack [2], puis l'utilisation d'un modèle de fondation, que nous avons affiné à nos données. Nous présenterons les résultats de ce fine-tuning et de la segmentation sur une vidéo.

Enfin, nous finirons par évoquer une conclusion et une discussion sur les perspectives de ce travail, suivies des matériels et méthodes utilisés, des références bibliographiques et des annexes.

# Chapitre 1

## Contexte biologique

### 1.1 Macrophages, cellules immunitaires dynamiques

, ...  
M1, M2, pro-inflammatoire, anti-inflammatoire  
polarisation,

### 1.2 Signal calcique, candidat prometteur

mehari, chen ...

### 1.3 Zebrafish, modèle animal idoine

lignée transgénique

### 1.4 Microscopie confocale

obtention des images, microscopie

Il faut au préalable réaliser toutes les manipulations telles que la reproduction de poissons, le triage des œufs, la sélection des larves, la coupe de la nageoire caudale en fonction de la condition que l'on cherche à observer

, l'observation et l'enregistrement microscopique pendant 40 minutes (durée des films que nous avons) à 3, 6, 24, 48 et 72 heures post-amputation, pour chacun des poissons sélectionnées et pour finir le traitement des films sorties de microscopie pour qu'ils soient exécutables. Cela demande un coût humain et financier énorme.

# Chapitre 2

## Segmentation automatique d'images microscopiques de macrophages

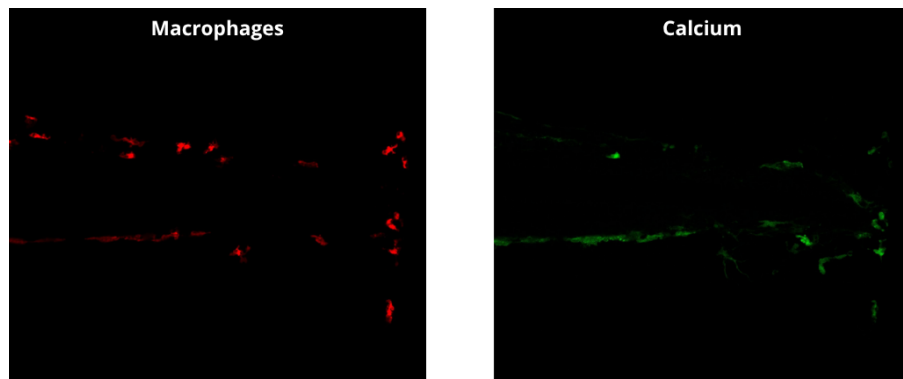
Nous avons pris en main un code existant, MacTrack [2], utilisant Kartezio, une librairie Python implémentant une méthode [3] de segmentation d'images de microscopie. Nous avons créé un nouveau jeu de données pour l'entraînement des modèles de segmentation produits par Kartezio. Une nouvelle version de MacTrack a été publiée et nommée MacTrack2 [4]. Elle implémente des scripts intuitifs, une documentation détaillée disponible sur un site internet, et fournit un jeu de données exemple à la fois pour la création de modèle mais aussi pour l'analyse d'un film (voir **Disponibilité du code**). Cette nouvelle version permet aux biologistes de l'équipe de traiter plus facilement leurs nouvelles acquisitions microscopiques.

### 2.1 Introduction à la segmentation d'images

#### 2.1.1 Image numérique

Les images que nous traitons sont obtenues par microscopie confocale de zebrafish, comme nous l'avons vu dans la section 1.4. La microscopie confocale nous permet d'obtenir deux types d'images en niveau de gris où chaque pixel est codé en 8 bits donc par un nombre entre 0 (noir) et 255 (blanc) : 1/ les images correspondant aux macrophages et 2/ les images correspondant à la présence de calcium.

Nous « colorons » artificiellement les images afin de différencier les macrophages des signaux calciques. Le choix des couleurs est fait en accord avec les longueurs d'onde des protéines fluorescentes utilisées pour le marquage. Ainsi, nous avons un canal rouge pour la présence des macrophages et un canal vert pour la présence de calcium dans le cytoplasme de ces cellules (Fig. 2.1). Nous parlerons ci-après de « canal rouge » et de « canal vert ». Il est important de noter que chaque image est une vue en deux dimensions de l'échantillon, parfaitement nette en tout point et en format 8 bits.



**Figure 2.1.** Images en 2-dimension de la nageoire blessée d'une larve de zebrafish réalisée par microscopie confocale.

L'image rouge montre les macrophages (fluorescence de la *mCherry*) et l'image verte montre le signal calcique (fluorescence de la protéine fluorescente *GCaMP6f*). Voir la Fig. A.1 pour la superposition des deux canaux.

### 2.1.2 Différentes approches de segmentation d'images

La segmentation d'images est un domaine de la vision par ordinateur (*Computer Vision* ou CV) qui consiste à diviser une image en plusieurs régions ou objets d'intérêt. Ici, les objets d'intérêt sont des macrophages, mais la segmentation est un problème plus général, applicable à de nombreux domaines. Les régions ou objets d'intérêts peuvent prendre plusieurs formes comme par exemple des rectangles permettant de délimiter la présence d'un objet. Elle sert à **détecter** tous les objets d'intérêt dans une image et peut même être utilisée pour labelliser ces objets [5, 6, 7, 8].

La **segmentation sémantique** est une autre approche de segmentation d'images où on affecte à chaque pixel une classe d'objet, et ce pour chaque objet de nature (ou sémantique) différente. Elle ne fait donc pas la différence entre deux voitures de couleur ou modèle différents par exemple, mais les classifieira chacun comme étant une voiture.

La **segmentation d'instance** est une approche plus complexe qui se concentre sur la distinction des instances, c'est-à-dire des objets individuels, même les instances occluses de la même classe d'objet. Si on reprend notre exemple, les voitures sont maintenant segmentées individuellement, même si elles sont identiques, puisqu'elles sont vues comme des objets différents mais appartenant à la même classe « voiture ».

La **segmentation panoptique** combine les deux précédentes. Elle englobe à la fois la classification sémantique de chacun des pixels de l'image et la délimitation de chaque instance d'objet, mais son coût de calcul est plus élevé.

Dans notre cas, nous cherchons à segmenter des macrophages, donc sur le canal rouge de l'image. Nous avons donc besoin d'une segmentation d'instances car nous voulons détecter chaque macrophage tout en faisant attention à ne pas les confondre entre eux.

Cependant, la segmentation d'instances fait appel à beaucoup de paramètres et des méthodes comme les réseaux de neurones convolutifs. Ces paramètres sont

optimisés via une fonction de perte, ce qui nécessite un apprentissage supervisé sur des milliers d'images annotées. Or, annoter à la main un grand nombre d'images nécessite beaucoup de temps et le nombre d'images que nous disposons au laboratoire est limité.

Parmi les modèles de classification sémantique, il avait été découvert l'année précédente *Kartezio* [3], une méthode développée en 2023 basée sur le *Cartesian Genetic Programming* (CGP) [9]. Elle permet de segmenter des objets dans des images de microscopie et présente de nombreux avantages.

### 2.1.3 Évaluation des performances

#### Intersection over Union (IoU)

Dans toute méthode de segmentation d'images, il est nécessaire d'avoir une métrique qui va nous permettre d'évaluer la qualité de l'apprentissage. La métrique la plus utilisée est l'**IoU** (*Intersection over Union*). On peut la définir comme étant le rapport entre l'intersection (en tant qu'aire) de la prédiction du modèle et de la vérité annotée et de leur union, pour un objet dans une image (Fig. 2.2) :

$$\text{IoU}(M_k, G_k) = \frac{|M_k \cap G_k|}{|M_k \cup G_k|} = \frac{\sum_{i,j} M_k[i, j] \cdot G_k[i, j]}{\sum_{i,j} M_k[i, j] + \sum_{i,j} G_k[i, j] - \sum_{i,j} M_k[i, j] \cdot G_k[i, j]}$$

avec :

- $i \in \{1, \dots, H\}$  et  $j \in \{1, \dots, W\}$  la hauteur et la largeur respectivement de l'image,
- $M_k = (M_k[i, j])_{i,j}$  le masque prédit pour l'objet  $k$ , une matrice binaire de taille  $H \times W$  où chaque pixel vaut 1 s'il est dans le masque prédit, c'est-à-dire que si la probabilité que le pixel  $(i, j)$  appartienne au masque de l'objet  $k$  est supérieure à 0.5, sinon il vaut 0. On a donc :

$$M_k[i, j] = \begin{cases} 1 & \text{si } p > 0.5, \\ 0 & \text{sinon.} \end{cases}$$

avec  $p$  la probabilité que le pixel  $(i, j)$  appartienne au masque de l'objet  $k$ ,

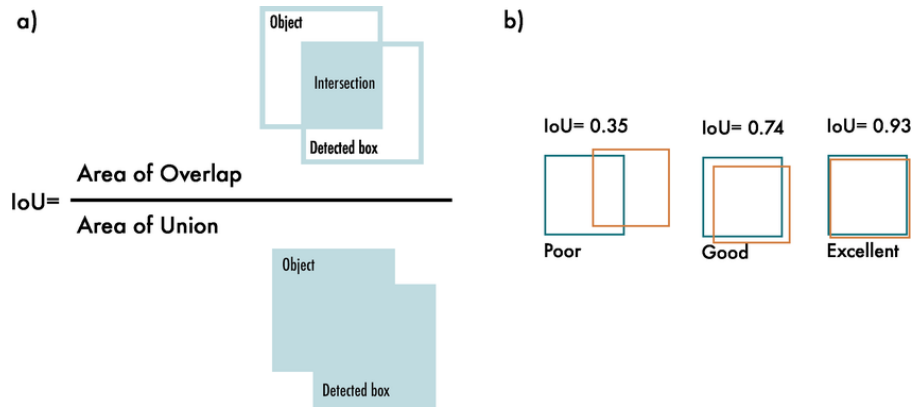
- $G_k = (G_k[i, j])_{i,j}$  le masque de vérité pour l'objet  $k$ , une matrice binaire de taille  $H \times W$  où chaque pixel (élément de la matrice) vaut 1 s'il est dans le masque de vérité, sinon il vaut 0,
- $|M_k \cap G_k| = \sum_{i,j} M_k[i, j] \cdot G_k[i, j]$  l'intersection entre le masque prédit et le masque de vérité, autrement dit le nombre de pixels en commun entre les deux masques,
- $|M_k \cup G_k| = \sum_{i,j} M_k[i, j] + \sum_{i,j} G_k[i, j] - \sum_{i,j} M_k[i, j] \cdot G_k[i, j]$  l'union entre le masque prédit et le masque de vérité, autrement dit le nombre de pixels dans au moins un des deux masques,

- $\sum_{i,j} M_k[i, j]$  le nombre de pixels du masque prédit,
- $\sum_{i,j} G_k[i, j]$  le nombre de pixels du masque de vérité.

Pour obtenir la valeur de l'IoU pour une image entière, on fait la moyenne des IoUs de tous les objets  $k$  segmentés dans l'image :

$$\text{mIoU}(M, G) = \frac{1}{K} \sum_{k=1}^K \text{IoU}(M_k, G_k) \quad (2.1)$$

où  $K$  est le nombre total d'objet dans une image,  $M = (M[i, j])_{i,j}$  le masque prédit pour l'image (sous la forme d'une matrice binaire de taille  $H \times W$ ) et  $G = (G[i, j])_{i,j}$  le masque de vérité pour l'image (sous la même forme que  $M$ ).



**Figure 2.2.** Schématisation de l'IoU (figure issue de [10]).

Il s'agit d'une valeur comprise entre 0 et 1, où 0 signifie que la prédiction ne recouvre pas du tout la vérité et où 1 signifie qu'elle la recouvre parfaitement. Plus on obtient une valeur d'mIoU proche de 1 et plus on peut en déduire que la prédiction est bonne. Cependant, c'est une métrique qui a tendance à sous-estimer la qualité de la segmentation. En effet, si un objet est mal segmenté mais qu'il recouvre une grande partie de la vérité, l'mIoU aura une valeur convenable alors que la segmentation n'est pas si spécifique au problème, on manquera alors de précision.

Dans la pratique, obtenir une segmentation parfaite, signifiant que l'mIoU vaudra 1, n'est tout simplement pas réaliste. Il est presque impossible que toutes les coordonnées  $(x, y)$  de la prédiction recouvrent toutes les coordonnées de la vérité. À travers l'IoU, on cherche donc à récompenser les prédictions qui se superposent bien avec la vérité.

Ainsi il est considéré, qu'une mIoU supérieure à 0.5 est une mIoU « acceptable » et on considère que si l'mIoU dépasse 0.7, il s'agit d'une « bonne » mIoU (voir [11, 12]). Cette valeur est souvent utilisée comme seuil pour différencier les vrais positifs des faux positifs ou faux négatifs. Par exemple, le jeu de données test *Pascal VOC* [13], qui est utilisé pour la détection d'objet et utilise l'mIoU comme

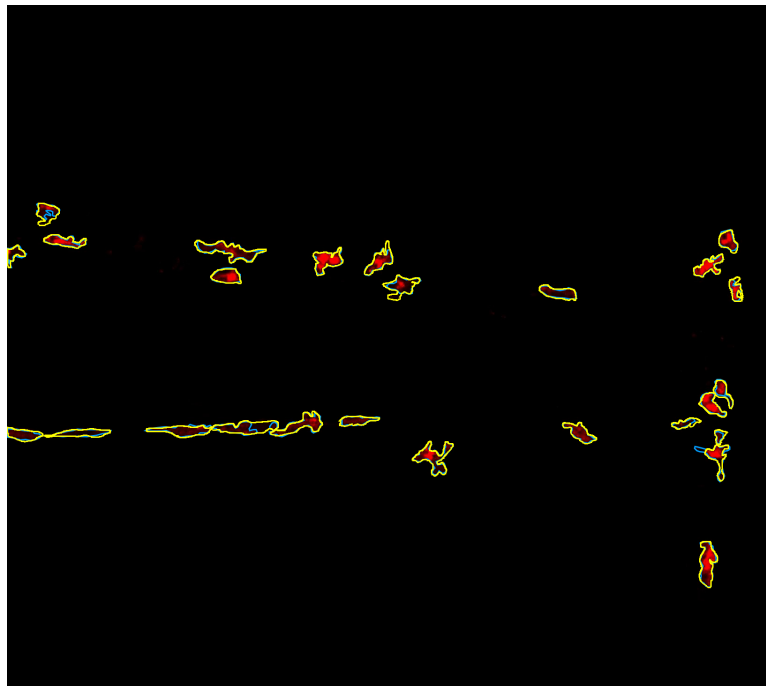


métrique principale, utilise un seuil de 0.5 ce qui signifie que toute détection d'objet ayant une mIoU supérieure à 0.5 est considérée comme une détection positive (vrai positif). Il s'agit d'un consensus commun dans la communauté de la CV, construit sur un compromis entre qualité visuelle et quantitative.

### **Différence de segmentation entre deux opérateurs.**

Les annotations d'images sont souvent sujettes à la subjectivité de chaque opérateur détournant les objets d'intérêt. En effet, deux opérateurs, souvent des experts dans le milieu d'application, peuvent avoir des opinions divergentes ce qui résulte en des annotations différentes. Dans notre cas, comme peut en témoigner la Fig. 2.1, les délimitations des macrophages (dans le canal rouge de l'image) sont parfois floues et peuvent porter à confusion. De plus, certains objets pourtant bien colorés en rouge peuvent être confondus pour des macrophages alors qu'il s'agit de pigments de coloration dus à l'auto fluorescence de ces derniers.

Nous avons donc voulu comparer les annotations de deux opérateurs, moi-même qui ai annoté l'entièreté des images utilisées pour entraîner les différents modèles, et une membre de l'équipe, qui a annoté quelques images. La Fig. 2.3 montre la superposition des annotations avec en jaune celles de ma collègue et en bleu les miennes. Nous avons utilisé le logiciel ImageJ [14] pour réaliser ces annotations. Pour plus de détails concernant l'utilisation de ce logiciel, voir la section 2.2.3.



**Figure 2.3.** Comparaison des annotations de deux opérateurs sur une image de macrophages.

Les annotations sont obtenus avec le logiciel ImageJ. Pour plus de détails sur l'utilisation de ImageJ, voir la section 2.2.3. Les annotations des deux opérateurs sont superposées sur l'image rouge (macrophages). En bleu, mes annotations et en jaune celles de ma collègue. Les opérateurs ne se sont pas concertés au préalable. Nous obtenons une mIoU de 0.8429 entre les deux annotations.

Image issue d'un film à 3h post-amputation de la nageoire caudale d'une larve de zebrafish.

Comme attendu, les délimitations des macrophages selon l'opérateur ne sont pas les mêmes, et il y a même des cas où certaines parties du macrophage sont annotées par l'un mais pas par l'autre. Cela se reflète dans la valeur de l'mIoU entre les segmentations des deux opérateurs. En effet, nous obtenons une mIoU de 0.8429. Cela signifie que nous nous rejoignons sur 84,29% des pixels annotés. En général, on considère que la différence de segmentation entre deux experts d'un domaine (que je ne suis pas) est de l'ordre de 15 à 30%.

Nous avons donc une marge d'erreur de 15% entre les deux annotations. De plus, nous avons effectué cette comparaison seulement pour une image. Il aurait fallu le faire pour toutes les images du jeu de données annoté. Cela nous aurait permis de quantifier la différence entre deux opérateurs et de remettre en question ou non les performances du modèle. En effet, si notre modèle a des performances similaires à la comparaison entre deux opérateurs humains, cela signifie que le modèle est capable de segmenter les macrophages avec la même variabilité qu'aurait un troisième opérateur humain.

## 2.2 Calibration de modèles de segmentation avec Mactrack2

La librairie MacTrack2 (voir **Disponibilité du code**) que nous avons développée en poursuivant le travail fourni par MacTrack [2] a la particularité de faciliter l'entraînement de modèles de segmentation d'images. Nous avons implémenté un script simple qui permet d'entraîner un modèle de segmentation d'images à partir d'un jeu de données annotées.

En effet, nous utilisons principalement Kartezio [3], une librairie Python implémentant une méthode de segmentation d'image basée sur le *Cartesian Genetic Programming* (CGP) [9].

### 2.2.1 Cartesian Genetic Programming (CGP)

Le CGP [9], initialement créé pour optimiser des circuits électroniques [15], est une approche de programmation génétique [16] (*Genetic Programming* (GP)) qui utilise des graphes pour coder des programmes informatiques (Fig. 2.4). La GP est un algorithme évolutif, c'est-à-dire qu'il s'inspire des éléments de la biologie évolutive (génotype, génome, phénotype, sélection, recombinaison ou *crossover*, reproduction, mutation, ...) pour résoudre des problèmes d'optimisation discrète complexes. Le terme « cartésien » du CGP vient du fait que les modèles générés sont représentés grâce à une grille de nœuds, où chaque nœud représente une fonction ou une opération. Chaque nœud est connecté à d'autres nœuds, formant ainsi un graphe orienté acyclique (Fig. 2.5). Le CGP est particulièrement adapté pour les problèmes de classification et de régression, mais il peut également être utilisé pour la segmentation d'images.

Chaque graphe peut être représenté par une chaîne d'entiers, qu'on appelle le génotype. Celui-ci est ensuite décodé pour obtenir le phénotype, qui est le programme exécutable. Le phénotype est obtenu en parcourant le graphe, en commençant par

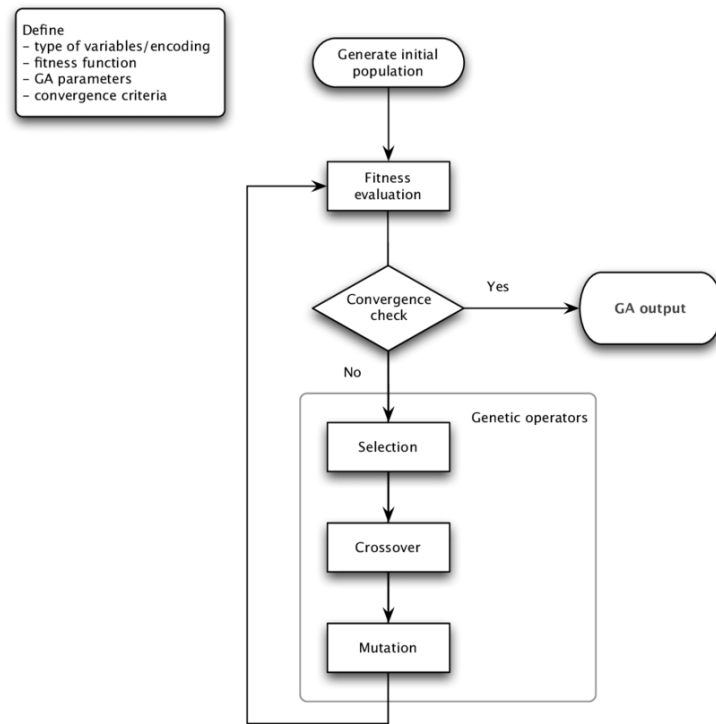
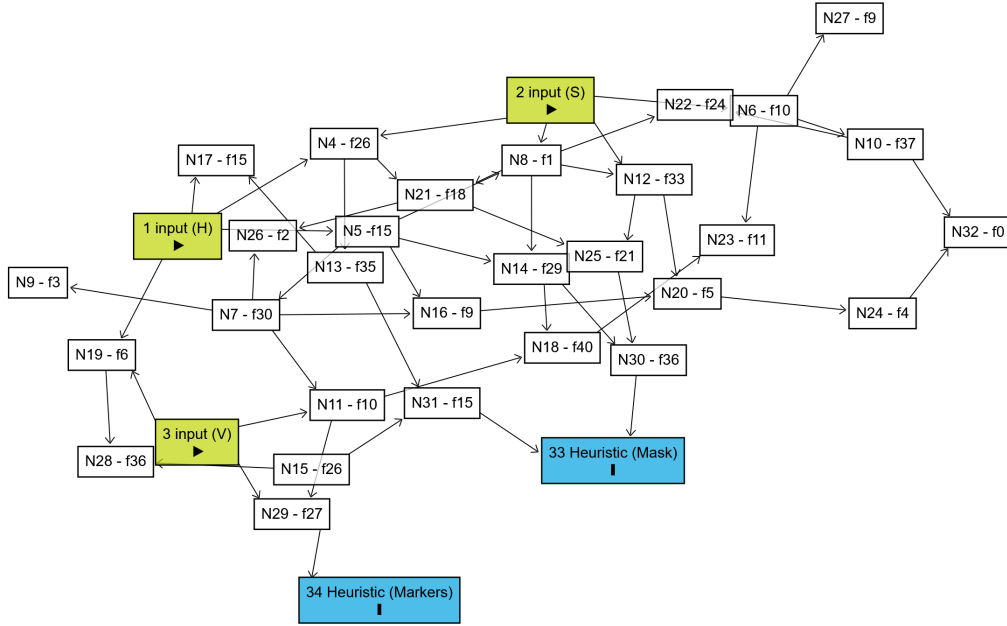


Figure 2.4

les nœuds d'entrée (*input*) et en appliquant les fonctions de chaque nœud aux entrées correspondantes.

Un modèle CGP utilise une stratégie évolutive (*evolutionary strategy* (ES)) qui va décider de quel opérateur génétique appliquer à chaque génération. Nous avons par exemple la stratégie  $(1 + \lambda)$  où un individu (*parent*) génère  $\lambda$  descendants (*offspring*) par mutation et le meilleur de ceux-ci, selon une fonction d'adaptation (*fitness*), est conservé et deviendra l'individu parent de la génération suivante. Si les performances du meilleur descendant sont inférieures à celles du parent, alors la descendance complète est rejetée. On parle de remplacement strict. On peut aussi avoir une stratégie de mutation ciblée, où on altère aléatoirement les gènes (nœuds), en modifiant les fonctions et les connexions, avec un taux de mutation ciblée. Il existe une stratégie de sélection par tournoi, où l'on compare des sous-ensembles d'individus et on sélectionne le plus performant de chacun des groupes, qui est ensuite utilisé pour la reproduction. Il existe des stratégies hybrides qui combinent le CGP avec des méthodes d'apprentissage automatique comme le Deep Learning.

Le CGP, contrairement à d'autres approches de GP, n'utilise pas vraiment de recombinaison (*crossover*) et privilégie plutôt des mutations ciblées sur sa population d'individus. De plus, le CGP a une propriété de neutralité génétique, c'est-à-dire que certains changements dans le génotype ne seront pas forcément des changements visibles dans le phénotype mais seront conservés. Cependant, ces nœuds servent à



**Figure 2.5.** Exemple de graphe orienté acyclique de notre propre modèle (voir section 2.2).

$Nx$  avec  $x$  le numéro du nœud et  $fy$  avec  $y$  le numéro de la fonction parmi une liste de fonctions (voir Tab. A.1 pour la numérotation des fonctions). Graphe obtenu avec Dagitty ([17]).

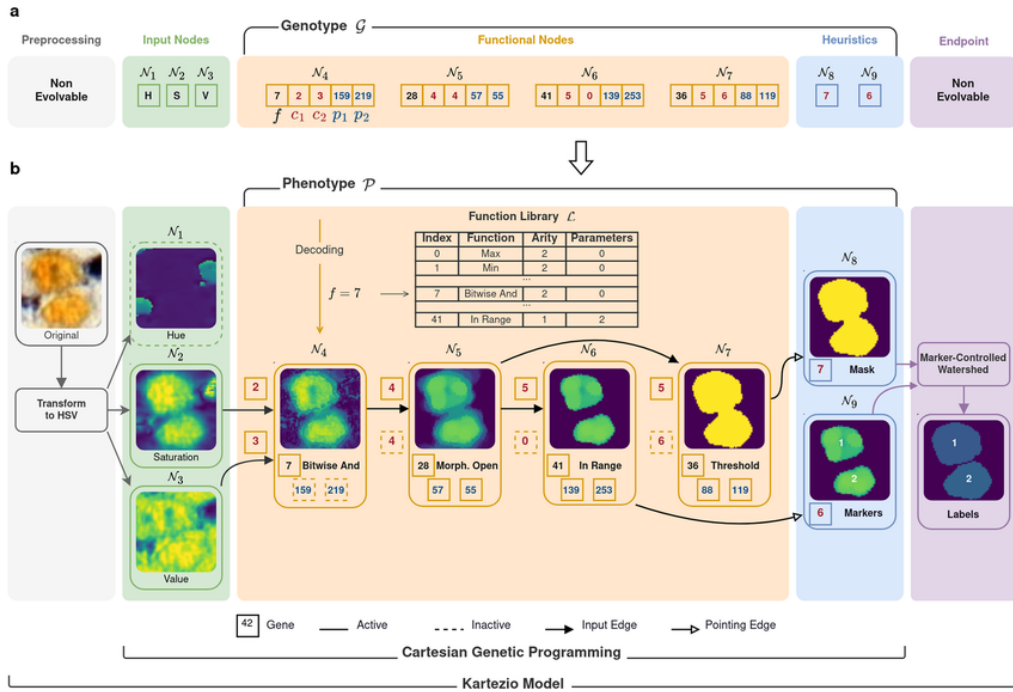
explorer l'espace de recherche d'une manière plus efficace. Certains nœuds, d'un autre côté, qui sont eux efficaces, peuvent être facilement dupliqués et réutilisés. Cela nous permet d'obtenir des solutions plus compactes en restant tout autant efficaces. Dans le cadre de notre problème, qui est la segmentation d'images, cette particularité du CGP est très utile, car souvent, en segmentation d'images, la même opération est souvent utilisée plusieurs fois dans différentes parties de l'image.

La stratégie évolutive la plus utilisée est la stratégie  $(1 + \lambda)$ , car elle favorise la simplicité et la stabilité du processus d'évolution, tout en restant efficace et en limitant la taille de la population. La mutation reste l'opérateur génétique principal utilisé dans cette stratégie. C'est la même stratégie évolutive qu'utilise *Kartezio* pour générer des pipelines de segmentation d'images biomédicales.

### 2.2.2 Kartezio

*Kartezio* [3] est une stratégie computationnelle de segmentation d'images biomédicales s'établissant sur le CGP [9]. Disponible sous la forme d'une librairie Python, cette méthode est capable de générer des pipelines de segmentation d'images qui sont à la fois clairs et interprétables, en assemblant et paramétrisant des fonctions connues de traitement d'images issues de la librairie *OpenCV* [18, 19]. La liste de fonctions issues de cette librairie utilisées par *Kartezio* est disponible en Annexe A.2 (Tab. A.1).

Comme introduit précédemment, Kartezio présente de nombreux avantages par rapport aux méthodes de segmentation d'images classiques. Tout d'abord, le nombre de données (images) annotées nécessaires pour entraîner les paramètres d'un modèle de segmentation est nettement réduit par rapport aux méthodes dites état de l'art que sont, par exemple, les modèles d'apprentissage profond (*Deep Learning*). Kartezio est donc une méthode que l'on appelle *Few-Shot Learning*, c'est-à-dire que l'on a besoin seulement de quelques images annotées pour entraîner un modèle qui ait des performances satisfaisantes. La Fig. 2.6 issue de l'article de Cortacero et al. [3] montre bien la structure de leur modèle, héritée du CGP.



**Figure 2.6.** Architecture d'un modèle généré par Kartezio. Figure issue de [3].

Dans un second temps, la méthode est efficace. L'article de Cortacero et al. [3] montre que Kartezio est capable de segmenter des images en entrant en concurrence avec les méthodes état de l'art basées sur des réseaux de neurone, telles que *Cellpose* [20, 21, 22], *Mask R-CNN* [23] ou encore *Stardist* [24].

Construire des modèles à l'aide de Kartezio est aussi plus rapide que les méthodes états de l'art. La Tab. 2.2 nous montre bien que Kartezio est capable de créer des modèles en moins de 10 minutes. Les méthodes classiques, au contraire, demandent un temps d'entraînement de l'ordre de plusieurs heures, voire plusieurs jours, pour obtenir des performances similaires.

Kartezio, par son utilisation du CGP, est aussi une méthode qui produit des modèles compréhensibles et interprétables. En effet, les modèles générés sont des graphes orientés acycliques et les fonctions utilisées dans chaque nœud sont issues d'une librairie de fonctions de traitement d'images connues. Cela nous permet donc de retracer le cheminement du pipeline de segmentation (voir Fig. 2.5 et Tab. A.1 pour avoir un exemple concret).

Kartezio a adapté le CGP pour la segmentation d'images, en utilisant les avantages de celui-ci afin de créer une méthode efficace et rapide. En effet, contrairement au CGP classique, Kartezio introduit le concept de nœuds non évolutifs (*non evolvable*, voir Fig. 2.6), qui sont le *preprocessing*, qui permet de convertir l'image originale en un format préférentiel, ici le format HSV (*Hue, Saturation, Value* ou TSV en français pour Teinte, Saturation et Valeur), qui constitue ensuite respectivement les 3 premiers nœuds d'entrée :  $\mathcal{N}_1$ ,  $\mathcal{N}_2$  et  $\mathcal{N}_3$ . Cela permet d'uniformiser les images d'entrées.

Ainsi, nous avons une méthode qui fonctionne pour tous les formats d'images classiques (png, jpg, ...) et même les plus flexibles comme le format TIFF (*Tag Image File Format*), qui est un format utilisé notamment en photographie pour le peu de perte de pixels qu'il engendre quand on modifie un tel fichier, mais aussi pour le fait qu'il supporte les images sur plusieurs couches comme les *z-stacks* qui sont beaucoup utilisés en microscopie confocale *in vivo*, comme c'est le cas pour nous.

La dernière opération du pipeline est nommée *Endpoint* et est l'application d'une fonction choisie par l'utilisateur qui permet de créer un arrêt intentionnel des processus d'optimisation pour que celui des nœuds d'heuristique prenne place. Il fait partie intégrante du processus évolutif du génotype puisqu'il agit comme un point d'arrêt où à chaque génération, on vérifie qu'il n'est pas dépassé (si c'est un seuil, par exemple), mais il n'est pas voué à évoluer.

L'efficacité, la rapidité, l'interprétabilité de Kartezio nous ont poussé à choisir cette méthode pour effectuer la segmentation automatique des macrophages dans des images de microscopie. Sachant que ce projet est destiné à être utilisé ultérieurement par les membres de l'équipe, il était primordial d'utiliser une méthode rapide, efficace, compréhensible et intelligible pour les biologistes.

### 2.2.3 En pratique

Nous allons maintenant décrire la phase d'apprentissage des modèles. Nous avons dans un premier temps besoin d'un jeu de données annotées, c'est-à-dire un ensemble d'images où les macrophages sont détournés à la main.

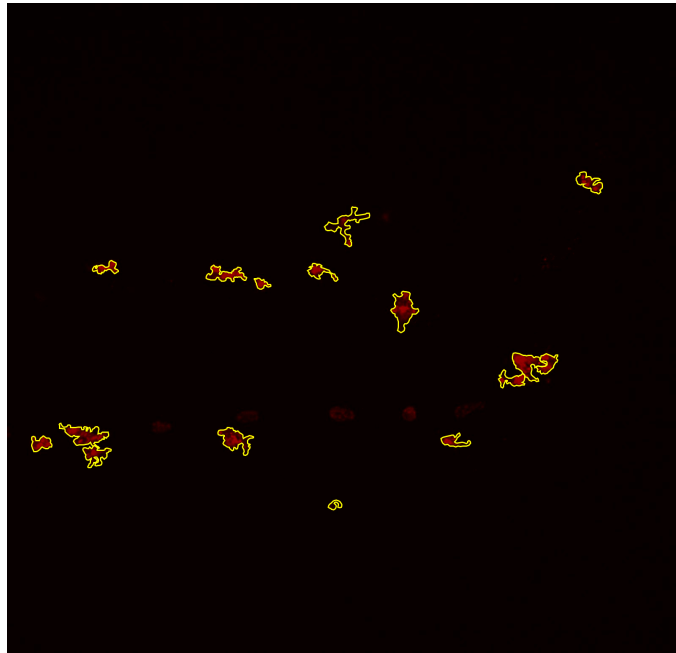
Ces images et leurs annotations, fournies dans un format spécifique, sont ensuite utilisées pour optimiser les paramètres du modèle de segmentation en minimisant une fonction de perte. Cette minimisation est faite par Kartezio, qui utilise une approche d'optimisation discrète.

#### Création d'un jeu de données d'entraînement

Nous avons utilisé ImageJ [14] pour annoter nos données. ImageJ est un logiciel permettant de modifier et analyser manuellement les images issues de microscopie. Il est communément utilisé par les biologistes.

En effet, les biologistes l'utilisent principalement pour réaliser une analyse quantitative des images obtenues par microscopie. Il est possible de compter le nombre de cellules dans une image par exemple ou de mesurer la taille, la surface, le périmètre ou l'intensité de fluorescence. Ce logiciel permet aussi d'améliorer la qualité des images en appliquant des filtres, des seuils ou encore une soustraction de l'arrière-plan. Dans notre cas, la possibilité de détourer des images est une fonction clé qui nous permet de produire et d'annoter les images dont nous avons besoin.

Nous avons donc pu détourer à la main chacun des macrophages dans les 25 images d'entraînement et 6 images de test que nous avons choisies aléatoirement parmi les différents films que nous disposions (Fig. 2.7).



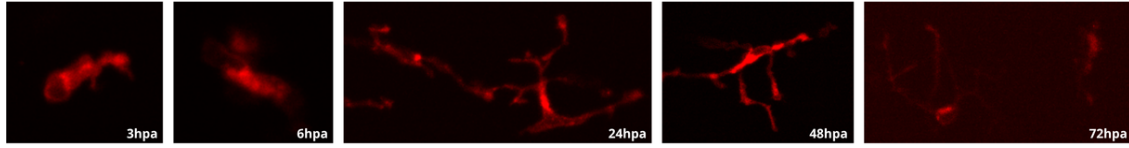
**Figure 2.7.** Image annotée issue du jeu de donnée d'entraînement créé.

Image issue d'un film à 3h post-amputation de la nageoire caudale d'une larve de zebrafish. Il s'agit de la première image du film. Les macrophages sont détourés à la main avec ImageJ. Les annotations sont représentées en jaune sur l'image.

Les annotations sont données sous la forme de fichiers ROI (Region Of Interest). Ces régions sont représentées dans la Fig. 2.7. Un détourage correspond à un macrophage, et le détourage de chaque macrophage dans l'image est enregistré dans un fichier ROI unique. Ainsi, chaque image annotée est accompagnée de plusieurs fichiers ROI. Ces fichiers sont ensuite compressés dans un fichier ZIP portant le nom de l'image. Nous avons donc un fichier ZIP par image, représentant les annotations de celle-ci.

Les images que nous avons choisies, provenant de films générés dans différentes conditions (nageoire coupée ou contrôle) et à des temps différents, sont représentatives de la diversité des images de macrophages que l'on peut obtenir. En effet, comme nous l'avons évoqué plus tôt, les macrophages peuvent être plus ou moins proches les uns des autres. De plus, ils ont des formes extrêmement variables en fonction de leur état de polarisation (Fig. 2.8). Par exemple, un macrophage dit M1 (pro-inflammatoire) aura tendance à être plutôt rond et compact, on peut l'observer à

3hpa (heures post-amputation), tandis qu'un macrophage M2 (anti-inflammatoire) aura tendance à être plus « filamenteux » et allongé, on peut l'observer à 24hpa.



**Figure 2.8.** Macrophages observés à différents temps post-amputation pour un même poisson.

Les macrophages sont observés, de gauche à droite, à 3, 6, 24, 48 et 72 heures post-amputation (hpa). Il s'agit d'un poisson dont la nageoire caudale a été coupée (ne faisait pas partie du groupe contrôle).

mettre la figure dans le contexte biologique et y faire référence ici aussi

## Préparatifs pour l'entraînement

Nous avons ensuite organisé le jeu de données dans un répertoire ayant une structure spécifique pour que Kartezio puisse le lire correctement. La structure du répertoire est la suivante :

```
dataset/
|-- train/
|   |-- train_x/
|   |   |-- 001_frame_crop_small.png
|   |   |-- 002_frame_crop_small.png
|   |   L-- ...
|   L-- train_y/
|       |-- 001_masks_crop_small.png
|       |-- 002_masks_crop_small.png
|       L-- ...
|-- test/
|   |-- test_x/
|   |   |-- 001_frame_crop_small.png
|   |   |-- 002_frame_crop_small.png
|   |   L-- ...
|   L-- test_y/
|       |-- 001_masks_crop_small.png
|       |-- 002_masks_crop_small.png
|       L-- ...
|-- dataset.csv
L-- META.json
```

Le fichier `dataset.csv` est un résumé des répertoires `train` et `test`. Il indique pour chaque image, le chemin vers l'image (*input*) et vers l'annotation ([*label*]). Une troisième colonne spécifie si l'image fait partie du jeu de données d'entraînement ou de test. La Tab. 2.1 nous montre un exemple de ce fichier.

Le fichier `META.json`, quant à lui, permet à Kartezio de savoir les formats des images (image HSV) et des annotations (ROI polygonaux).



| input                                  | label                                  | set      |
|----------------------------------------|----------------------------------------|----------|
| :                                      | :                                      | :        |
| train/train_x/024_frame_crop_small.png | train/train_y/024_masks_crop_small.zip | training |
| train/train_x/025_frame_crop_small.png | train/train_y/025_masks_crop_small.zip | training |
| train/train_x/001_frame_crop_small.png | train/train_y/001_masks_crop_small.zip | testing  |
| train/train_x/002_frame_crop_small.png | train/train_y/002_masks_crop_small.zip | testing  |
| :                                      | :                                      | :        |

**Table 2.1.** Extrait du fichier `dataset.csv` nécessaire pour la reconnaissance des images (*input*) et des annotations (*label*) pour Kartezio, selon leur appartenance (*set*) au jeu d'entraînement (*training*) ou de test (*testing*).

Il s'agit là d'un extrait du fichier `dataset.csv` pour le modèle « small+cropped » présenté dans la section 2.3.3.

Ainsi, le jeu de données est finalisé et prêt à l'emploi pour l'apprentissage des modèles de segmentation d'images.

### Calibration du modèle (entraînement)

Une fois le jeu de données d'entraînement sauvegardé dans le répertoire `DATASET` = `"examples/input_model/dataset"`. L'entraînement se lance avec une commande simple.

```
from kartezio.dataset import read_dataset
from kartezio.training import train_model

dataset = read_dataset(DATASET)      # Chargement du jeu de données
↳ d'entraînement par Kartezio
elite, a = train_model(model, dataset, OUTPUT,
↳ callback_frequency=frequency)    # Entraînement du modèle
```

Nous avons choisi les hyperparamètres servant à l'entraînement du modèle de la manière suivante.

```
from kartezio.apps.segmentation import create_segmentation_model
from kartezio.endpoint import EndpointThreshold

generations = 1000      # Nombre de générations à entraîner
_lambda = 5              # Nombre d'individus (offspring) par génération
frequency = 5            # Fréquence d'affichage des résultats (en
↳ générations)
rate = 0.1               # Taux de mutation des nœuds et des sorties

model = create_segmentation_model(
    generations,
    _lambda,
    inputs=3,             # Nombre d'entrées du modèle (3 pour le
↳ format HSV)
```

```

nodes=30,                # Nombre de nœuds évolutifs dans le modèle
node_mutation_rate=rate,  # Taux de mutation des nœuds
output_mutation_rate=rate, # Taux de mutation des sorties
outputs=1,                # Nombre de sorties du modèle (1 pour la
    ↪ segmentation binaire)
fitness="IOU",            # Fonction de fitness utilisée pour
    ↪ l'optimisation
endpoint=EndpointThreshold(threshold=4), # Point d'arrêt (Endpoint)
    ↪ pour l'optimisation
)

```

Nous allons décrire brièvement les hyperparamètres qui ont un impact significatif sur l'entraînement du modèle.

Nous réalisons 1000 générations lors de cet entraînement, avec  $\lambda = 5$  nouveaux individus (descendants) générés à chaque génération par le parent. Plus on augmente le nombre de générations et de nouveaux individus par génération, et plus le temps de calcul augmente. La valeur de  $\lambda$  est fixée à 5 car il s'agit de la version de la stratégie évolutive  $1 + \lambda$  la plus utilisée en pratique.

Nous avons fixé le nombre de nœuds évolutifs à 30 dans ce modèle. En essayant de faire varier ce nombre, nous n'avons pas observé d'amélioration significative dans les performances. Les nœuds évolutifs sont les seuls à évoluer lors de l'entraînement, à travers des mutations dont le taux a été fixé à 0.1. Ils correspondent à la partie rosée dans la Fig. 2.6.

La fonction de perte utilisée pour l'optimisation est l'IoU, notée  $\mathcal{L}_{IoU}$ . Nous avons défini précédemment l'IoU, dans la section 2.3, comme étant la métrique principale pour évaluer la qualité de segmentation. La fonction de perte associée  $\mathcal{L}_{IoU}$  est définie comme suit :

$$\mathcal{L}_{IoU} = 1 - IoU$$

Cette perte est minimale quand l'IoU est maximale, c'est-à-dire quand la prédiction recouvre parfaitement la vérité terrain (annotations). Optimiser cette fonction de perte revient donc à chercher une segmentation qui maximise la similarité entre la prédiction et la vérité.

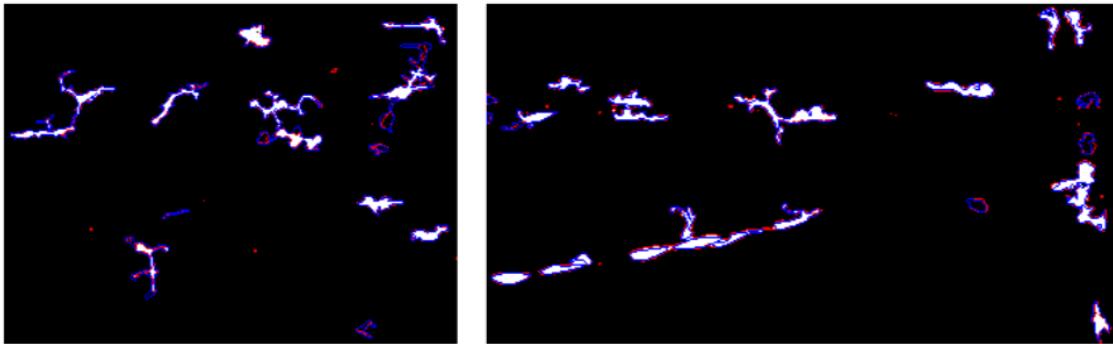
Cet entraînement prend entre quelques minutes et plus d'une heure, dépendamment du format des images, comme nous le présentons dans la section 2.3.3. Les images de plus grande taille demandent plus de temps de calcul car nous traitons les images pixel par pixel.

De plus, la queue du poisson à l'imagerie microscopique ne prend pas la totalité du champ d'acquisition, ce qui signifie que nous avons des zones (en haut et en bas de l'image) qui sont vides, avec des pixels noirs. Ces parties sont quand même traitées et cela a un impact sur le temps de calcul.

## 2.3 Résultats

### 2.3.1 Visualisation de la qualité de segmentation

Nous avons mis en place un outil permettant de visualiser la qualité des modèles entraînés. En superposant les prédictions données par le modèle et la vérité sur les images ayant permis d'obtenir ce dernier, nous pouvons visualiser la qualité de la segmentation qu'a le modèle que nous venons de créer. La Fig. 2.9 nous montre, pour une image venant de l'échantillon d'entraînement et une venant de l'échantillon de test, la superposition des prédictions du meilleur modèle, celui écrit en rouge dans la Tab. 2.2, et de la vérité annotée.



**Figure 2.9.** Exemples de comparaisons entre la vérité annotée (en bleu) et les prédictions du modèle choisi (en rouge) sur des images issues de l'échantillon d'entraînement (à gauche) et de test (à droite).

Le modèle choisi est celui écrit en rouge dans la Tab. 2.2. Pour l'image d'entraînement, l'IoU est de 0.67 et pour l'image de test, l'IoU est de 0.76. Ce sont toutes les deux des images issues du jeu de données « petit+crop ».

Les prédictions en rouge, obtenues en appliquant le modèle sur chacune des images des deux échantillons, sont parfois très petites en aire, on peut observer des petites taches considérées comme macrophages. Il est possible que le modèle, entraîné pour détecter des niveaux de gris sur les images, détecte du bruit de fond ou des artefacts créés par le microscope.

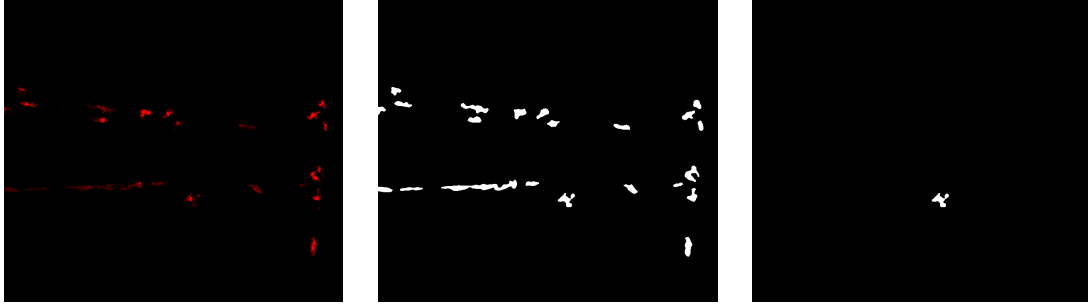
C'est pour cela que la préparation des images au préalable est une étape primordiale à ne pas négliger. Préparées par l'équipe sur le logiciel ImageJ, les images ont été soigneusement traitées pour faire en sorte d'avoir des images avec un fond uniforme, dont l'intensité moyenne est proche de 0. Ce traitement permet d'améliorer la qualité des images en entrée du modèle et donc la performance des algorithmes de segmentation.

### 2.3.2 Segmentation automatique d'images avec MacTrack2

Notre librairie MacTrack2 est capable, à partir des modèles entraînés, de segmenter des images. En effet, la fonction `locate` permet de le faire. À partir d'une image, on effectue la prédiction du modèle sur celle-ci pour obtenir une première segmentation. Nous appliquons un filtre de taille sur les objets détectés afin de laisser de côté les objets trop petits qui ne sont pas des macrophages. Souvent, il s'agit

d'artefacts que l'on obtient lors de la prise d'image et ne sont pas visibles à l'œil nu sur l'image d'origine.

Nous séparons aussi chacun des objets détournés pour les représenter dans des images différentes. Nous pouvons ainsi observer exclusivement la segmentation de chacun des macrophages. La Fig. 2.10 nous montre les différentes sorties de la fonction `locate`. À gauche, nous retrouvons l'image d'origine, au milieu la segmentation obtenue après application du filtre de taille et à droite la segmentation d'un des macrophages détournés.



**Figure 2.10.** Illustrations des différentes sorties de la fonction `locate`

Nous retrouvons à gauche l'image d'origine, au milieu la segmentation complète après application du filtre de taille des objets détectés et à droite la segmentation objet par objet. Ces images proviennent de l'observation de la nageoire caudale d'un poisson à 3h post-amputation. Il s'agit de la deuxième image du film.

### 2.3.3 Performance/Résultat Comparaison des modèles selon le format d'images

Pour des questions de performances et de temps de calculs, nous avons choisi de créer 4 jeux de données issus des mêmes images mais avec des tailles différentes. Dans le premier, *modèle initial*, l'image n'a pas été modifiée, c'est-à-dire de la taille de l'image originale. Dans le second, le modèle *small*, nous avons divisé la taille de l'image par 4, réduisant ainsi le nombre de pixels à traiter par image.

Pour le troisième, nous avons retiré, dans les images, les zones noires dans lesquelles il n'y a pas d'objets à segmenter car ces zones ne font pas partie du poisson (il s'agit seulement du champ d'observation du microscope). Nous avons nommé celui-ci le modèle *cropped*. Dans le dernier cas, nous avons combiné les modifications d'images du deuxième et du troisième jeu de données, donnant ainsi le modèle *small+cropped*.

C'est avec ce dernier que nous avons obtenu les meilleurs résultats dans des temps record. Nous conservons donc ce dernier pour générer des modèles. La Tab. 2.2 nous donne une idée des gains entre les différents jeux de données.

Le meilleur modèle obtenu a un score d'mIoU de 0.712 sur l'échantillon d'entraînement et de 0.763 sur l'échantillon de test. En comparaison avec les modèles obtenus l'année précédente sur un jeu de données différent, le modèle retenu avait un score d'mIoU de 0.80 sur l'échantillon d'entraînement et de 0.63 sur l'échantillon de test.

| Modèle <i>initial</i> |             | Modèle <i>small</i> |             | Modèle <i>cropped</i> |             | Modèle <i>small+cropped</i> |              |
|-----------------------|-------------|---------------------|-------------|-----------------------|-------------|-----------------------------|--------------|
| <b>Train</b>          | <b>Test</b> | <b>Train</b>        | <b>Test</b> | <b>Train</b>          | <b>Test</b> | <b>Train</b>                | <b>Test</b>  |
| 0.659                 | 0.663       | 0.639               | 0.628       | 0.674                 | 0.706       | <b>0.712</b>                | <b>0.763</b> |
| 0.647                 | 0.619       | 0.646               | 0.581       | 0.698                 | 0.735       | 0.687                       | 0.755        |
| 0.634                 | 0.639       | 0.637               | 0.634       | 0.682                 | 0.681       | 0.716                       | 0.727        |
| Temps de calcul       |             |                     |             |                       |             |                             |              |
| <b>Train</b>          | <b>Test</b> | <b>Train</b>        | <b>Test</b> | <b>Train</b>          | <b>Test</b> | <b>Train</b>                | <b>Test</b>  |
| 1h27m                 | 1s          | 36m30s              | 0s          | 49m48s                | 1s          | 9m0s                        | 1s           |
| 1h28m                 | 2s          | 5m39s               | 0s          | 1h                    | 2s          | 4m6s                        | 0s           |
| 1h54m                 | 3s          | 7m35s               | 0s          | 1h44m                 | 2s          | 3m48s                       | 0s           |

**Table 2.2.** Exemples de performances des modèles entraînées sur les différents jeux de données.

Les valeurs sont les IoU moyens sur le jeu de données d'entraînement et de test. Le modèle dont les performances sont écrites en gras est celui que nous avons conservé pour la suite, et que nous avons fourni en exemple dans le dépôt GitHub du projet. Les différents modèles sont le modèle *initial* avec les images telles quelles, le modèle *small* où le nombre de pixel dans les images est divisé par 4, le modèle *cropped* où les zones noires en haut et en bas des images qui ne sont pas des parties du poisson ont été retirées, et le modèle *small+cropped* qui combine les deux derniers modèles en divisant par 4 le nombre de pixels dans les images du modèle *cropped*. Les temps de calcul sont données en heures (h), minutes (m) et secondes (s).

Nous avons donc réussi à obtenir un modèle qui est meilleur que celui de l'année dernière sur l'échantillon de test. Cela peut s'expliquer par la diversité des données que nous avons sélectionnées pour constituer notre jeu de données. En effet, nous avons choisi des images provenant de différents poissons, à différents temps post-amputation. Nous avons même choisi des images provenant de poissons contrôle dont la nageoire caudale n'a pas été coupée. Nous avons donc un modèle plus généraliste et nous avons évité le sur-ajustement.

De plus, nous avons automatisé le processus de création de modèle à travers des scripts et un jeu de données exemple que l'on fournit dans le dépôt GitHub du projet (voir **Disponibilité du code**). Donner un tel jeu de données permet à l'utilisateur de comprendre les exigences structurelles de Kartezio, qui peuvent être fastidieuses pour des personnes non aguerries.

# Chapitre 3

## Suivi temporel des macrophages

Les données que nous traitons sont des vidéos qui sont un empilement d’images. Ces vidéos sont des observations microscopiques d’une quarantaine de minutes, où une image est prise toutes les 9 secondes, résultant en un film de 266 images.

Nous allons commencer par étendre la segmentation sémantique obtenue par la librairie MacTrack2 à l’ensemble des images de la vidéo. Nous allons ensuite appliquer un suivi temporel, réalisé l’allée précédente, sur chacun d’eux afin de suivre chacun des macrophages de la vidéo. Nous obtiendrons donc un tracking des macrophages, c’est-à-dire un suivi de chacun des macrophages présents dans la vidéo, nous rapprochant ainsi de la segmentation d’instances.

L’apport de MacTrack2 est le post-traitement qui en suit. Nous verrons comment nous arrivons à reconstruire entièrement la segmentation d’un objet.

D’un autre côté, nous allons utiliser un modèle de fondation afin de traiter la vidéo directement et non pas image par image. Ce modèle étant généraliste, il nous sera nécessaire de l’adapter à notre problème en modifiant ses paramètres à travers un *fine-tuning*.

### 3.1 Tracking à partir de la segmentation d’images

Une fois le modèle généré, visualisé et validé, il est possible de l’appliquer sur des vidéos entières et pas seulement sur une image. Lorsque les vidéos traitées par l’équipe sortent du logiciel, elles sont au format AVI et ont 266 images. Pour plus de détails sur l’obtention des vidéos, voir **Matériel et Méthodes**.

Nous commençons donc par convertir les vidéos des canaux rouge et vert au format MP4. Puis, nous découpons cette vidéo en images, les plaçons dans des dossiers spécifiquement créés pour respecter la structure nécessaire au bon fonctionnement de Kartezio, comme nous l’avons vu dans la section 2.2.3. Nous créons automatiquement les fichiers `dataset.csv` et `META.json` pour conserver cette structure.

Nous avons donc une structure de dossiers qui est similaire à celle que nous avons utilisée pour l’entraînement des modèles. Cependant, il y a une différence notable. En effet, lors de l’entraînement, nous avons les images servant à la validation du modèle dans le dossier « test ». Ici, nous devons placer les images issues de la vidéo dans ce dossier, mais il n’y a pas de vérité terrain associée à ces images.

Kartezio ne regarde pas les contenus des annotations pour segmenter les images, mais demande quand même qu'elles soient présentes dans la structure des dossiers. Ainsi, nous avons créé des fichiers ZIP vides pour chaque image de la vidéo dans le dossier « test\_y ».

Il s'agit d'une nomination de dossier qui peut prêter à confusion car il ne s'agit pas d'un véritable test du modèle. Cependant, c'est la structure requise au bon fonctionnement de Kartezio.

Nous avons aussi besoin d'un fichier `dataset.csv`, où un extrait est donné par la Tab. 2.1. Ce fichier indique le chemin des images, qu'elles viennent de l'entraînement ou de la vidéo à segmenter ainsi que les masques associés même s'ils sont vides. Finalement une colonne « set » indique si l'image est une image d'entraînement ou de la vidéo.

Les paragraphes qui suivent présentent brièvement les différentes étapes constituant le tracking des macrophages à partir de la segmentation d'images. Il s'agit du fruit du travail de l'année précédente, ayant abouti à la librairie MacTrack [2]. Une version plus détaillée de cette partie est disponible en Annexe A.3.

### Segmentation d'une vidéo.

La segmentation de la vidéo commence après l'étape préparatoire présentée ci-dessus. La fonction `locate` est appliquée image par image, produisant une première segmentation de l'ensemble des images de la vidéo. Nous avons donc des images segmentées en noir et blanc ainsi que des dossiers individuels contenant la segmentation de chaque macrophage par image (Fig. 2.10).

Cependant, la segmentation sémantique peut regrouper plusieurs macrophages proches en un seul objet. De plus, la microscopie confocale, par projection de stacks d'images en une seule, peut créer des occlusions où un macrophage en cache un autre. Ces cas compliquent le suivi des macrophages.

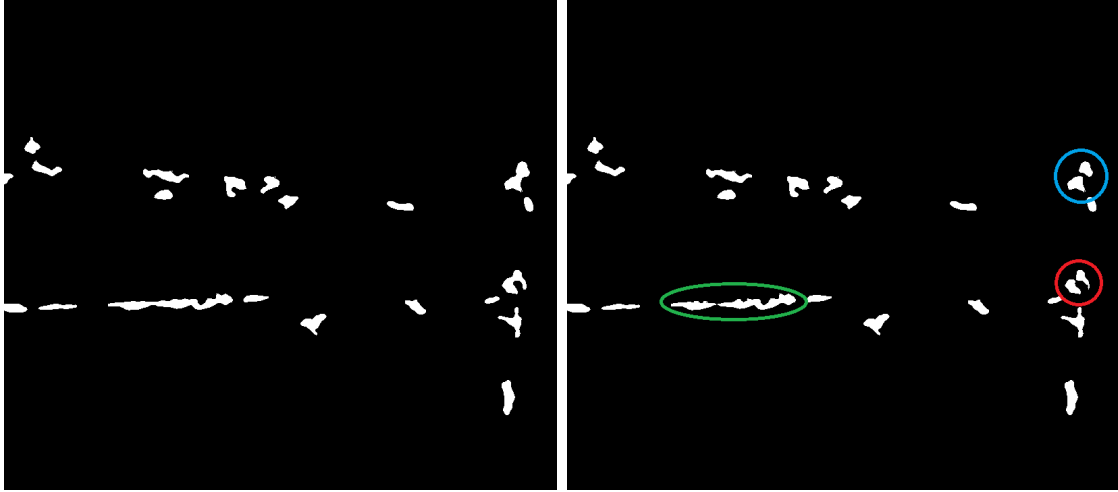
Pour y remédier, un post-traitement est réalisé afin de corriger et d'affiner la segmentation initiale, garantissant un tracking plus précis.

### Correction de la segmentation.

Les macrophages ont tendance à se regrouper, ce qui complique leur segmentation. Lorsqu'ils sont très proches, la distinction entre eux peut être difficile, même pour les experts. Ce n'est donc pas anodin que la première segmentation puisse les fusionner.

Pour corriger cela, nous utilisons la dimension temporelle des vidéos via une fonction appelée `defuse`. Elle compare chaque image à la précédente en calculant l'IoU entre les macrophages. Si un macrophage dans l'image actuelle correspond à la fusion de plusieurs macrophages de l'image précédente (IoU supérieure à 0 entre un macrophage de l'image actuelle et plusieurs macrophages de l'image précédente), on applique une séparation basée sur la segmentation antérieure, afin de défaire cette fusion.

Cette correction s'applique aussi dans le sens inverse, de l'image suivante à l'image actuelle, ce qui permet de traiter à la fois les fusions et les séparations erronées. Ce post-traitement a grandement amélioré la qualité du suivi en rendant le nombre de macrophages détectés plus fidèle à la réalité (Fig. 3.1).



**Figure 3.1.** Exemple d'application de la fonction `defuse`.

À gauche, la segmentation d'une image avec plusieurs macrophages fusionnés avec un autre. À droite, la segmentation après application de la fonction `defuse` qui a séparé les deux macrophages pour les 3 cas. Il s'agit de la 14ème image d'un film de 266 images. L'acquisition a été faite à 3h post-amputation de la nageoire caudale d'un poisson.

Cependant, cette méthode peut réduire la précision des contours des macrophages et augmenter le temps de calcul, chaque image est traitée en fonction de l'image précédente et suivante. Il faut donc considérer ces compromis selon la longueur de la vidéo.

### Tracking des macrophages.

Après correction de la segmentation, chaque image contient une liste de macrophages sous forme de masques binaires. Pour les suivre dans le temps, nous utilisons la fonction `track`, qui compare chaque image actuelle avec la précédente.

Pour chaque macrophage de l'image actuelle, on calcule l'IoU avec ceux de l'image précédente. Si l'IoU dépasse 0.5, ils sont considérés comme identiques et partagent un même identifiant. Sinon, un macrophage nouveau ou disparu est détecté.

La correction préalable a évité les fusions erronées, rendant le suivi très précis. Aucun problème de tracking n'a été rencontré.

### Choix du seuil d'IoU.

Le seuil d'IoU de 0.5 a été choisi pour assurer un suivi efficace des macrophages. Ce seuil permet d'éviter de confondre deux macrophages distincts lorsque leurs segmentations se chevauchent légèrement d'une image à l'autre, surtout après la correction de la segmentation.



Ce choix tient compte de la forte mobilité des macrophages, en faisant un compromis entre précision du suivi et variations naturelles de position. Par ailleurs, une IoU de 0.5 est généralement considérée comme une prédiction correcte, ce qui renforce la pertinence du seuil.

### Photoblanchiment du microscope et seuil sur la durée de tracking.

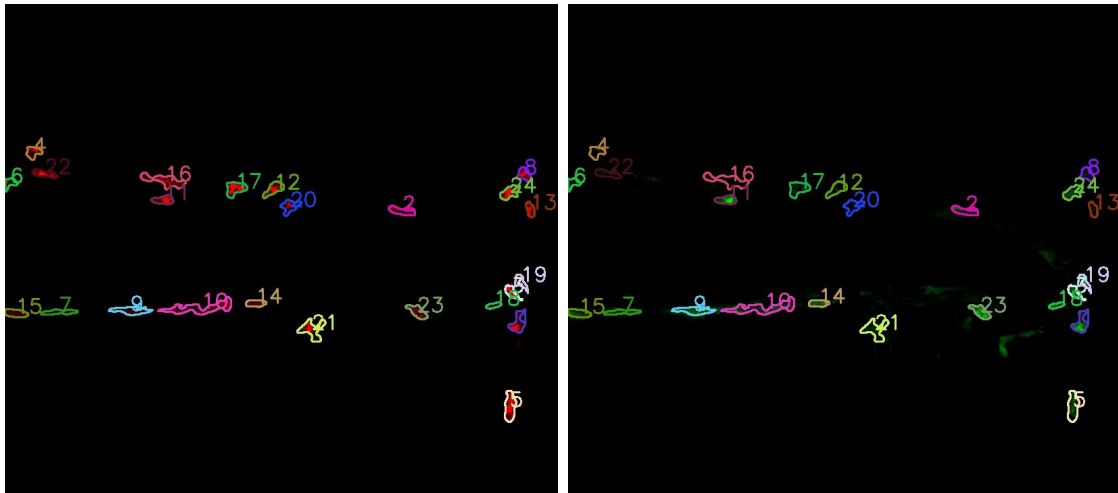
Le suivi des macrophages peut être affecté par le photoblanchiment, phénomène où la fluorescence diminue avec l'expression prolongée du laser, entraînant la disparition de certains macrophages.

Pour éviter de considérer comme macrophages des objets détectés sur très peu d'images, un seuil a été fixé : un macrophage doit être visible sur au moins 10 images consécutives (environ 1 minute 30 secondes) pour être pris en compte. Ce choix, issu de plusieurs essais, permet d'obtenir un nombre de macrophages suivi proche des estimations des experts selon les conditions expérimentales.

### Résultats et visualisation du tracking des macrophages

Pour visualiser le suivi des macrophages, la fonction `result` a été développée. Elle ajoute sur chaque image de la vidéo d'origine une numérotation et un contour coloré unique pour chaque macrophage détecté. Ce suivi est également reporté sur la vidéo du canal vert (calcium).

La Fig. 3.2 illustre cet affichage sur les deux canaux, montrant clairement les macrophages numérotés et délimités. Des exemples vidéos des segmentations avant et après post-traitement sont disponibles dans la section **Liens utiles**.



**Figure 3.2.** Résultats du tracking des macrophages sur une image de chaque canal de la vidéo.

À gauche, nous avons le résultat du tracking sur le canal rouge de la vidéo. À droite, nous avons le résultat du tracking sur le canal vert de la vidéo. Les macrophages sont numérotés et les contours sont dessinés avec une couleur unique par macrophage. Les images proviennent de la vidéo d'un poisson dont la nageoire caudale a été amputée et observée à 3h post-aputuation. Il s'agit de la première image du film.

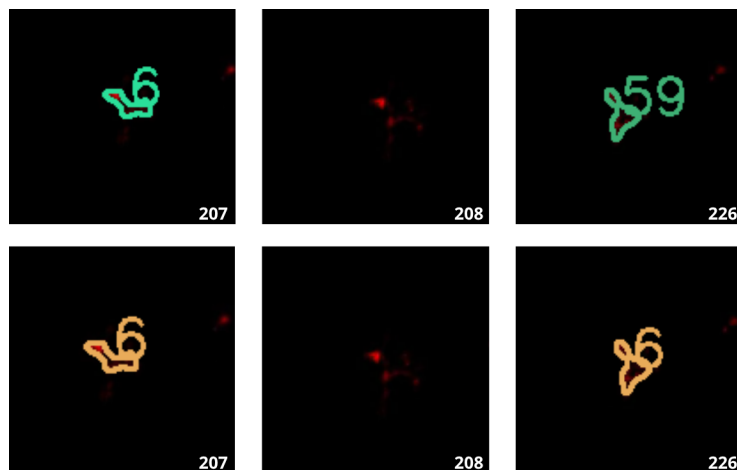
## 3.2 Post-traitement

Il est important que le suivi des macrophages soit continu afin de réaliser des analyses futures sur l'évolution de leur comportement. Cependant, il est possible que certains macrophages soient perdus pendant un temps, puis réapparaissent. Puisque nous n'avons pas de mémoire temporelle sur l'ensemble des images de la vidéo, si un tel macrophage existe, il sera considéré comme deux macrophages distincts. Il est donc important de regarder la vidéo en sortie dans son ensemble pour vérifier si la qualité de tracking est suffisante.

Nous avons donc mis en place, à travers la contribution de MacTrack2, un tracking fonctionnel des macrophages à travers le temps, avec l'instauration de filtres. Cela permet de se rapprocher du consensus biologique en terme de nombre de macrophages détectés.

La fonction permettant de fusionner les suivis de macrophages, visiblement identiques mais séparés, est nommée `merge`. Elle permet de fusionner les segmentations de différents macrophages en un seul. Elle permet de traiter différents cas de figure en agissant dans les dossiers contenant la segmentation de chaque macrophage à chaque image de la vidéo (obtenus à l'issue de la fonction `track`). En déplaçant certaines images d'un macrophage (sous-dossier) à un autre, on effectue la fusion de ces deux. Elle fait aussi en sorte de traiter les cas où la segmentation n'est pas continue ou alors si elle chevauche la segmentation d'un autre.

Dans le premier cas, si un macrophage est perdu pendant quelques images, puis réapparaît, alors la fonction `merge` va fusionner les deux segmentations et créer des images entièrement noires pour les images perdues.



**Figure 3.3.** Illustration du premier cas de fusion manuelle des macrophages.

Sur la première ligne, nous avons les images de la vidéo avant le traitement. Sur la seconde ligne, nous avons les résultats du traitement. Les nombres en bas à droite de chaque image correspondent aux numéros des images de la vidéo. Les images sont issues de la vidéo d'un poisson dont la nageoire caudale a été amputée et observée à 3h post-amputation.

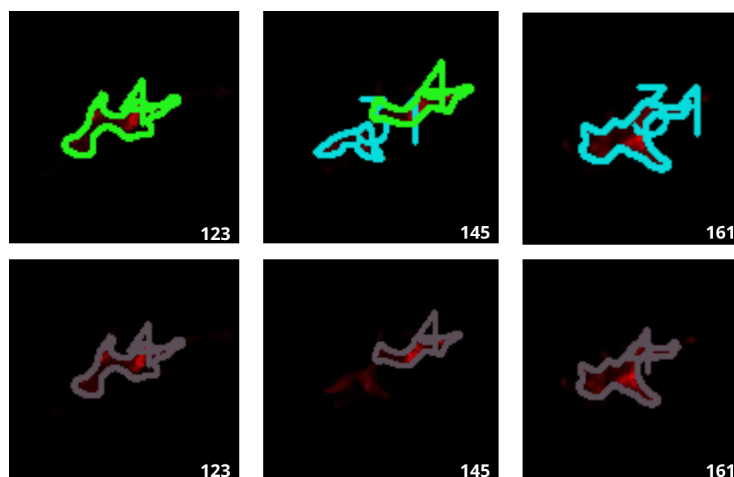
cut tail infection 3hpa, ligne du haut avant en bas apres

Le second cas apparaît souvent dans le cas où un macrophage est segmenté en tant que deux macrophages distincts pendant quelques images. Ainsi, avec la fonction `merge`, nous allons fusionner les deux segmentations. Pour les images où il y a deux segmentations différentes, nous allons garder celle qui apparaît en premier dans les paramètres de la fonction, en supprimant l'autre définitivement.

```
merge(macro_list=[4,31])
```

En effet, la fonction prend en paramètre une liste d'identifiants de macrophages à fusionner. Si dans le second cas il s'agit des macrophages 4 et 31 qui doivent être fusionnés, comme le montre la Fig. 3.4 alors sur les images où les deux apparaissent, nous allons garder la segmentation du macrophage 4 car il apparaît en premier dans la liste paramètre de la fonction `merge`.

Il s'agit là d'un compromis que nous avons réalisé entre avoir un tracking sur l'ensemble de la vidéo, ce qui améliorera nos études par la suite, et une petite perte d'information.

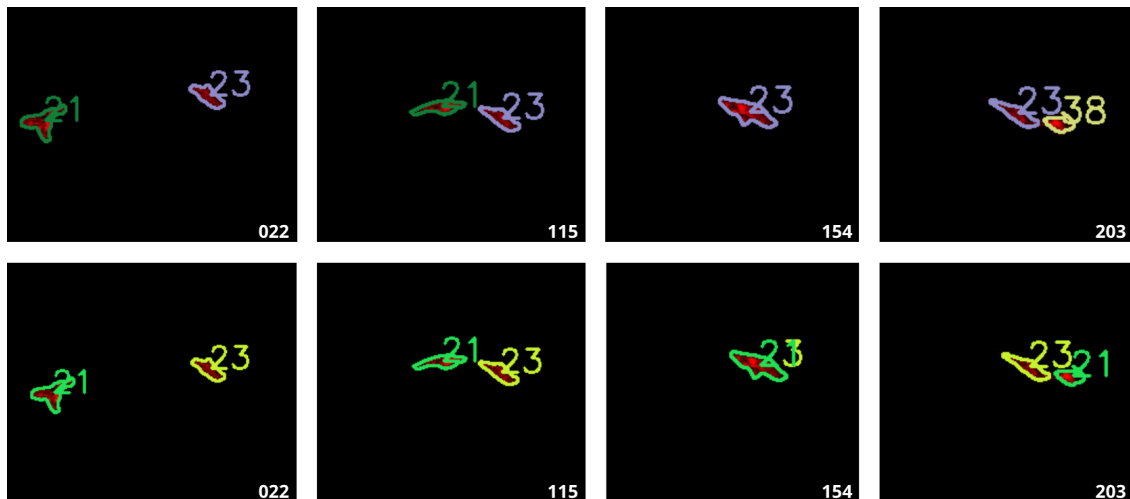


**Figure 3.4.** Illustration du second cas de fusion manuelle des macrophages.  
cut tail infection 3hpa, ligne du haut avant en bas apres

Comme nous l'avons vu plutôt, la microscopie se fait en prenant des images à différentes profondeurs, on parle de *z-stacks*. Il est donc possible que deux macrophages se chevauchent, que l'un « passe à travers » de l'autre. En réalité, il passe soit au dessus soit en-dessous d'un autre macrophage. Nous travaillons sur des données de dimensions 2D+t obtenues en faisant une projection des images 3D+t. Nous voyons donc sur la vidéo d'origine un macrophage qui fusionne avec un autre et qui se détache de lui quelques images plus tard.

Nous avons donc implémenté la fonction `merge_and_keep` qui permet, comme avec la fonction `merge`, de fusionner les segmentations de macrophages tout en gardant la segmentation du macrophage traversé, qui serait perdue sinon. En dédoublant la segmentation du macrophage traversé dans le dossier du macrophage traversant, nous obtenons donc une segmentation partagée entre les deux macrophages. La Fig. 3.5 nous montre un exemple de ce phénomène.

```
merge_and_keep(macro_list=[21,23,38], keep=[23])
```



**Figure 3.5**

cut tail 3hpa, ligne du haut avant en bas apres

graphiques a faire et changer les exemples dans le texte pour coller avec les graphes

après l'analyse petite conclusion : on a fait une segmentation framewise de vidéos mais ça reste super rapide et avec des bonnes performances, facilement modifiable et traitable par tout le monde ... pas évident (prise en main (temps, exigence de la structure), accessibilité (docu)) et efficace pour une première étape  
on peut maintenant se poser la question de ce qui suit, cad segmenter des vidéos en ayant une mémoire temporelle

## Chapitre 4

# Étude et reparamétrisation d'un modèle de fondation

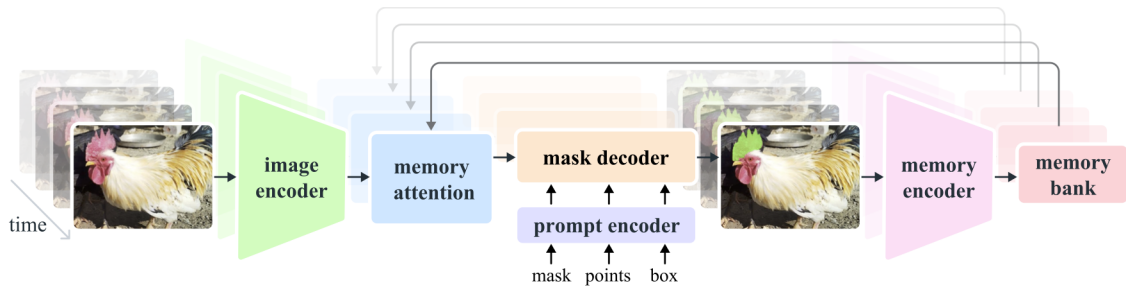
**nouvelle partie** De plus, nous avons une difficulté en plus car jusqu'à présent, nous avons seulement traité de segmentation d'images et non de vidéos. Or, les macrophages étant des cellules immunitaires très mobiles, ils pourraient se regrouper ou se séparer à tout moment de la vidéo. Donc la qualité de la segmentation est primordiale pour pouvoir au mieux retranscrire les comportements réels des macrophages.

### 4.1 Introduction

SAM-2 (*Segment Anything Model 2*) est un modèle de segmentation d'images et de vidéos développé par Meta AI **sorti récemment, en 2024**. Il s'agit d'une version améliorée de la première version du modèle, SAM [25], généralisant le problème de segmentation d'images au domaine des vidéos (*VOS : Video Object Segmentation*). Entraîné sur plusieurs jeux de données dont un premier de 50 000 vidéos, comprenant 600 000 masques (annotations). Les vidéos et masques utilisées pour entraîner un tel modèle proviennent du dataset SA-V [26] créé spécialement pour ce modèle et disponible sur demande. Les annotations ont été à la fois réalisées à la main par des humains mais aussi en étant assisté par la première version de ce modèle, SAM [25] puisqu'elles se font sur des images fixes et qu'il s'agit de la spécialité de la première version (190 000 annotations manuelles et 450 000 annotations automatiques), ce qui accélère le processus d'annotation qui est chronophage et demandeur en personnel. Ces vidéos ont été obtenues par un sous-traitant engagé par Meta FAIR pour récolter des vidéos dans 47 pays différents dans des situations du monde réel. Le second est nommé « *internal dataset* » pour jeu de données de vidéos disponibles sous licence en interne et Meta AI est moins clair sur l'obtention de ces données. Il comprend 62 900 vidéos pour 69 600 annotations. Il a permis d'étendre le jeu de données d'entraînement. Ainsi, SAM-2 est considéré (au même titre que SAM) comme un modèle de fondation (*foundation model*) [27], c'est-à-dire un modèle pré-entraîné sur une grande quantité de données, capable de généraliser à de nombreuses tâches différentes. Il est conçu pour être utilisé comme une base pour des tâches de segmentation en gardant de bonnes performances pour l'ensemble des

applications de segmentation d'images et de vidéos.

SAM-2 est capable de segmenter des objets dans des vidéos de manière précise et efficace. L'architecture du modèle (voir Fig. 4.1) est une architecture basée sur les Transformers [28]. Elle comprend un *image encoder* qui est un Vision Transformer hiérarchique appelé Hiera [29], un *prompt encoder* qui prend en compte des clics (positifs ou négatifs), des boîtes ou des masques permettant de définir un objet d'intérêt dans une image et un *mask decoder* qui prend ces entrées et en ressort un masque de segmentation de l'objet sélectionné. Il utilise une mémoire en flux (*streaming memory*). Celle-ci permet de traiter les images d'une vidéo en temps réel, une par une. Ainsi, il peut garder en mémoire, à l'aide de la *memory attention* et de la *memory bank*, les objets et leurs interactions au cours du temps, permettant ainsi de générer des masques plus précis et de les corriger en fonction des images précédentes. La *memory attention* permet de « préparer » l'image en prenant en compte la *memory bank*, qui garde en mémoire les précédentes images, mais aussi les nouveaux points, masques, boîte, que l'utilisateur peut ajouter à chaque image.



**Figure 4.1.** Architecture de SAM-2 (figure issue de [30]).

Un des problèmes que nous rencontrons avec SAM-2 est que l'utilisateur doit, à la main, sélectionner des objets d'intérêt dans une ou plusieurs images et si durant la vidéo un nouvel objet apparaît, il ne sera pas détecté tant que l'utilisateur ne l'a pas indiqué. Ainsi, nous allons utiliser la génération automatique de masques mise à disposition pour détecter les macrophages de manière automatique sur des images.

De plus, SAM-2 est un modèle de fondation, conçu pour être utilisé dans différents cas. Cependant, si on veut obtenir de bonnes performances dans un domaine spécifique comme le nôtre, il est nécessaire de faire un *fine-tuning*, c'est-à-dire un réentraînement des poids du modèle sur un jeu de données spécifique de notre domaine. Nous allons ainsi fine-tuner SAM-2 pour qu'il puisse mieux détecter les macrophages dans nos vidéos de microscopie.

## 4.2 Fine-tuning de SAM-2

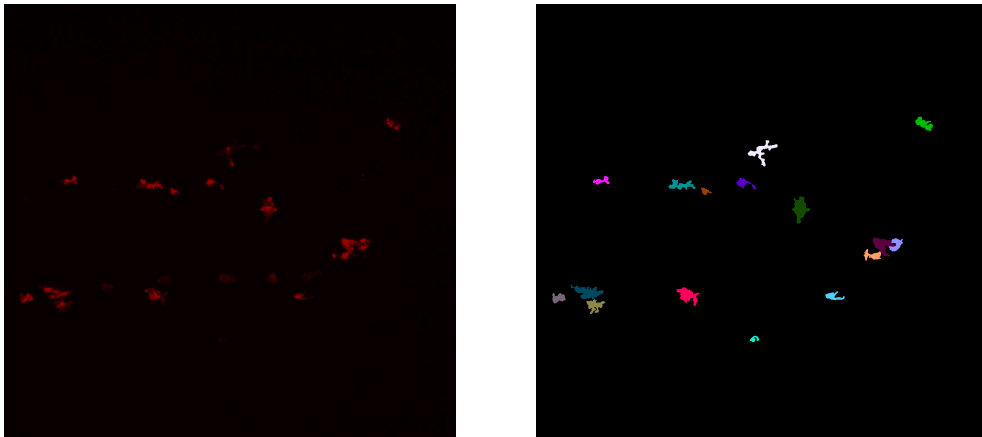
### 4.2.1 Définition

Le *fine-tuning* (ou *affinage* en français) est une technique d'apprentissage supervisé qui consiste à modifier les poids d'un modèle pré-entraîné sur une tâche spécifique. Elle est souvent utilisée pour adapter un modèle généraliste à une tâche particulière. Ici, SAM-2 a été entraîné sur des photos qui n'ont rien à voir avec la problématique biologique que nous rencontrons. Par conséquent, afin d'obtenir une bonne segmentation des macrophages, il est nécessaire de modifier les poids du modèle pour obtenir une version spécialisée dans la détection de macrophages. Cela nous évite de devoir entraîner notre propre modèle en partant de rien, ce qui nous demanderait beaucoup trop de données que nous n'avons pas. Mais le nombre de données nécessaires pour obtenir un bon affinage n'est pas négligeable.

Le fine-tuning de SAM-2 agit seulement sur les poids du *prompt encoder* et du *mask encoder*, c'est-à-dire les parties du modèle qui prennent en entrée les points, masques, boîtes, et qui génèrent les masques de segmentation. Nous spécifions donc la prise en charge des entrées données par l'utilisateur, mais pas la partie qui encode l'image ni la partie qui mémorise les images précédente une fois le fine-tuning fini et que l'on cherche à segmenter une vidéo entière.

### 4.2.2 Jeu de données

Les annotations sont sous la forme d'images où chaque macrophage est détourné à la main sur le logiciel *ImageJ* (et plus particulièrement *Fiji* [31]). Les détournages sont ensuite séparés par label, afin de s'assurer qu'un macrophage ne soit pas confondu avec un autre (Fig. 4.2).



**Figure 4.2.** Image d'origine (à gauche) comprenant des macrophages et leur détournage (à droite).

Malgré que le fine-tuning de SAM-2 ne nécessite pas autant de données que l'entraînement d'un modèle de segmentation complet, il est quand même nécessaire de disposer d'un nombre conséquent de données. Ici, nous avons besoin d'images et de

leur segmentation manuelle. Or, le temps imparti pour ce stage ne nous permet pas de composer un jeu de données complet qui satisferait ces besoins. Par conséquent, nous avons utilisé le programme MacTrack2 qui nous a permis de segmenter une vidéo de macrophages. Ainsi, nous disposons de 297 images et masques de macrophages (dont 266 faisant partie d’une seule vidéo) pour ré-entraîner les poids du modèle.

### 4.2.3 hyperparamètres

Les hyperparamètres, dans un modèle, constituent les paramètres non appris par le modèle durant l’entraînement (ou le ré-entraînement). Ils sont fixés à l’avance et influencent son comportement et par extension ses performances. Les hyperparamètres du pré-entraînement et de l’entraînement complet de SAM-2 sont disponibles dans leur papier [30] page 19.

Pour le fine-tuning, les sous-modules affectés sont le *mask decoder* et le *prompt encoder*. Nous utilisons l’optimiseur AdamW [32] avec un taux d’apprentissage (*learning rate*) de 0.0001 et une dégradation des poids (*weight decay*) de 0.00001, utile pour la régularisation L2. Le nombre total d’itérations est de 20 000, avec un nombre de pas pour l’accumulation des gradients (*accumulation step*) avant leur mise à jour égal à 4 steps. Le *scheduler* permet d’ajuster le *learning rate* durant l’entraînement pour améliorer la convergence du modèle. Ici nous utilisons un *stepLR scheduler*, ce qui signifie que tous les  $k$  steps, le *learning rate* est multiplié par un facteur  $\gamma$ . Nous avons  $k = 500$  et  $\gamma = 0.1$ . Ainsi, le *learning rate* est réduit de 90% tous les 500 pas.

Contrairement à ce que l’on peut retrouver habituellement dans l’entraînement de modèles, la taille du batch est égale à une seule image. En effet, à chaque pas ...

### 4.2.4 Fonctions de perte

Pour entraîner un modèle, il est nécessaire de définir une fonction de perte  $\mathcal{L}$  à minimiser. Elle est utilisée pour quantifier la qualité des prédictions du modèle par rapport à la « vérité » (ground truth). Dans le cas du fine-tuning de SAM-2, nous utilisons une combinaison de deux fonctions de perte : la *binary cross-entropy loss* qui sert de perte pour la segmentation et la *score loss* qui est utilisée en complément pour évaluer la qualité des prédictions des masques à partir de la métrique IoU (*Intersection over Union*). L’IoU est une métrique grandement utilisée dans les problèmes de segmentation d’images. Elle est définie dans la partie 2.1.3 et peut être illustrée par la Fig. 2.2.

En moyennant sur tous les objets d’une image, on obtient la mIoU (IoU moyenne), comme obtenue par (2.1) dans la partie 2.1.3.

Ainsi, nous pouvons définir la *score loss* comme suit :

$$\mathcal{L}_{\text{score}} = \frac{1}{K} \sum_{k=1}^K |s_k - \text{IoU}(M_k, G_k)|$$



avec  $K$  le nombre total de masques (donc de macrophages ou d'objets) dans l'image et  $s_k$  le score prédit pour l'objet  $k$  à partir de  $M_k$ .

Nous comparons donc le score prédit par le modèle pour chaque masque à la véritable qualité de chacun d'eux (l'IoU). Puisque'on cherche à minimiser cette perte, on aimerait que pour tout  $k$ ,  $s_k \simeq \text{IoU}(M_k, G_k)$  pour que la perte soit proche de 0.

Si  $K = 0$ , alors cela signifie qu'il n'y a pas de masques dans l'image, donc pas de macrophages. Un tel cas peut être rencontré, mais nous avons décidé de ne pas l'inclure dans le fine-tuning, car il n'apporte pas d'information utile concernant la segmentation des macrophages.

D'un autre côté, la *binary cross-entropy loss*  $\mathcal{L}_{BCE}$  est une fonction de perte généralement utilisée dans des problèmes de classification binaire, et notamment dans les problèmes de segmentation d'images. Elle est utilisée pour quantifier la différence entre les prédictions du modèle et les véritables annotations (masques de vérité). Elle est définie comme suit :

$$\mathcal{L}_{BCE} = -\frac{1}{K \times H \times W} \sum_{k=1}^K \sum_{i=1}^H \sum_{j=1}^W y_{k,i,j} \log \hat{y}_{k,i,j} + (1 - y_{k,i,j}) \log(1 - \hat{y}_{k,i,j})$$

avec :

- $y_{k,i,j} \in \{0, 1\}$  la valeur du pixel  $(i, j)$  du masque de vérité pour l'objet  $k$ . On peut notamment remarquer que la matrice  $G_k = (y_{k,i,j})_{i,j}$  est une matrice binaire (composée de 0 et de 1) de même dimension que les dimensions de l'image ( $H \times W$ ).
- $\hat{y}_{k,i,j} \in [0, 1]$  est la probabilité que le pixel  $(i, j)$  appartienne au masque de l'objet  $k$ . Elle est obtenue par la transformation sigmoïde appliquée à la prédiction des masques du modèle, qui sont des logits générés par le *mask decoder* (voir Tab. 4.1). On a donc :

$$\hat{y}_{k,i,j} = \sigma(z_{k,i,j}) = \frac{1}{1 + \exp(-z_{k,i,j})}$$

avec  $z_{k,i,j}$  le logit du pixel  $(i, j)$  du masque prédit pour l'objet  $k$ .

- $H$  et  $W$  la hauteur et la largeur de l'image.
- $K$  le nombre total de masques (donc de macrophages ou d'objets) dans l'image.

Interprétations fonction de perte ... ressemble a kullbackleibler mais différences ...

Finalement, la fonction de perte totale est une combinaison de ces deux dernières :

$$\mathcal{L} = \mathcal{L}_{BCE} + \lambda \mathcal{L}_{\text{score}}$$

avec  $\lambda$  un hyperparamètre qui permet de pondérer l'importance de la *score loss* par rapport à la *binary cross-entropy loss*. Dans notre cas, nous avons fixé  $\lambda$  à 0.05.

| Logit ( $z_{k,i,j}$ ) | Interprétation                                 | Sigmoid ( $\hat{y}_{k,i,j}$ ) |
|-----------------------|------------------------------------------------|-------------------------------|
| +10                   | Très sûr qu'il s'agit d'un objet               | $\approx 1$                   |
| +1                    | Assez confiant qu'il s'agit d'un objet         | $\approx 0.75$                |
| 0                     | Incertitude totale                             | 0.5                           |
| -1                    | Assez confiant qu'il s'agit du fond de l'image | $\approx 0.25$                |
| -10                   | Très sûr qu'il s'agit du fond de l'image       | $\approx 0$                   |

**Table 4.1.** Différents exemples des valeurs que peuvent prendre les sorties du *mask decoder* (logits), leur interprétation et la valeur de la probabilité associée (par transformation sigmoïde).

### 4.2.5 Optimisation

Une fois la perte calculée, elle est divisée par le nombre de pas d'accumulation des gradients, soit 4 dans notre cas, comme défini dans la partie 4.2.3 introduisant les hyperparamètres. Cela nous permet d'éviter de multiplier le gradient total par ce facteur. C'est utilisé lorsque l'on accumule des gradients sur plusieurs images (ici 4 fois une image) pour que la somme finale corresponde en réalité à une moyenne sur ce batch étendu. Ainsi, nous simulons un batch de taille 4 alors qu'en réalité nous n'avons qu'une seule image par step.

Ensuite, les gradients sont calculés en utilisant la méthode *backward* du package PyTorch [33], à partir de la perte totale  $\mathcal{L}$  divisée par cet *accumulation step*. Puis, tous ces 4 steps, nous mettons à jour l'optimiseur AdamW. Le nombre de pas avant la mise à jour du *learning rate* est de 500. Ainsi, le *learning rate* est réduit de 90% tous les 500 pas. Ensuite, les gradients sont remis à zéro pour le prochain pas d'accumulation.

Pour finir, nous calculons l'IoU moyenne au pas  $\ell$  comme étant :

$$\text{mIoU}_\ell = \begin{cases} 0.9 \cdot \text{mIoU}_{\ell-1} + 0.1 \cdot \frac{1}{K} \sum_{k=1}^K \text{IoU}(M_k, G_k) & \text{si } k > 0, \\ 0 & \text{sinon.} \end{cases}$$

### 4.2.6 Résultats et comparaisons

graphes miou et loss comp avant apres fine tune, performances et visuel

## 4.3 Propagation dans une vidéo

## Chapitre 5

# Analyse des signaux calciques issues de la segmentation des macrophages

extraction des données a partir de la segmentation (mactrack pour l'instant  
sam2 plus tard on espere)  
le faire pour plusieurs vidéos

# Chapitre 6

## Conclusion et perspectives

### 6.1 Disponibilité du code

Le code de MacTrack2 est disponible, ainsi que deux scripts détaillés pour la création de modèle et l'analyse d'une vidéo, avec un jeu de données d'exemple pour chacun des deux scripts, à l'adresse suivante : <https://github.com/guibouland/MacTrack2>.

Un site internet est disponible sur ce dépôt GitHub, regroupant toute la documentation et des instructions détaillées pour les utilisateurs plus novices, notamment les biologistes traitant la même problématique, qui sont la cible principale de ce projet. Il servira aussi aux futurs membres du laboratoire LPHI qui souhaitent utiliser MacTrack2 pour leurs propres projets.

### 6.2 Liens utiles

Liens vers les vidéos de résultats de segmentation dropbox ou lien similaire

# Chapitre 7

## Matériel et Méthodes

### Acquisition des images

partie contexte biologique

### Format des vidéos

reconstitution sur imagej... par norma images toutes les 9sec donc sur 40 min de prise de vidéo on a que 266 images

### Entraînement des modèles Kartezio

Les modèles Kartezio ont été entraîné sur le serveur de calcul Oban de l'Université de Montpellier, avec des processeurs possédant 32 cœurs et et 64 Go de RAM.

### Fine-tuning de SAM-2

L'optimiseur AdamW a été utilisé avec comme hyperparamètres un taux d'apprentissage (*learning rate*) de 0.0001, une dégradation de pondération (*weight decay*) de 0.0001. L'entraînement a été effectué avec 20000 itérations. Nous avons utilisé un GPU Nvidia A10 Tensor Core avec 24 Go de mémoire.

# Bibliographie

- [1] Fitsumbirhan T. MEHARI et al. « Intravital calcium imaging in myeloid leukocytes identifies calcium frequency spectra as indicators of functional states ». In : *Science Signaling* 15.744 (2022). DOI : [10.1126/scisignal.abe6909](https://doi.org/10.1126/scisignal.abe6909).
- [2] Axel de MONTGOLFIER. *Github - Stage LPHI 2024*. URL : <https://github.com/Axeldmont/Stage-LPHI-2024>.
- [3] Kévin CORTACERO et al. « Evolutionary design of explainable algorithms for biomedical image segmentation ». In : *Nature Communications* 14.1 (2023), p. 7112.
- [4] Guillaume BOULAND. *Github - MacTrack2*. URL : <https://github.com/guibouland/MacTrack2>.
- [5] Qing JIANG et al. *T-Rex2 : Towards Generic Object Detection via Text-Visual Prompt Synergy*. 2024. arXiv : [2403.14610](https://arxiv.org/abs/2403.14610) [cs.CV]. URL : <https://arxiv.org/abs/2403.14610>.
- [6] Tianhe REN et al. *Grounded SAM : Assembling Open-World Models for Diverse Visual Tasks*. 2024. arXiv : [2401.14159](https://arxiv.org/abs/2401.14159) [cs.CV].
- [7] Shilong LIU et al. « Grounding dino : Marrying dino with grounded pre-training for open-set object detection ». In : *arXiv preprint arXiv :2303.05499* (2023).
- [8] Tianhe REN et al. *Grounding DINO 1.5 : Advance the "Edge" of Open-Set Object Detection*. 2024. arXiv : [2405.10300](https://arxiv.org/abs/2405.10300) [cs.CV].
- [9] Julian MILLER et Andrew TURNER. « Cartesian Genetic Programming ». In : *Proceedings of the Companion Publication of the 2015 Annual Conference on Genetic and Evolutionary Computation*. GECCO Companion '15. Madrid, Spain : Association for Computing Machinery, 2015, p. 179-198. ISBN : 9781450334884. DOI : [10.1145/2739482.2756571](https://doi.org/10.1145/2739482.2756571).
- [10] Juan TERVEN et Diana-Margarita CORDOVA-ESPARZA. *A Comprehensive Review of YOLO : From YOLOv1 to YOLOv8 and Beyond*. Avr. 2023. DOI : [10.48550/arXiv.2304.00501](https://doi.org/10.48550/arXiv.2304.00501).
- [11] Gaundez BOESCH. *What is Intersection over Union (IoU) ?* <https://viso.ai/computer-vision/intersection-over-union-iou/>. 2024.
- [12] Zhaowei CAI et Nuno VASCONCELOS. *Cascade R-CNN : Delving into High Quality Object Detection*. 2017. arXiv : [1712.00726](https://arxiv.org/abs/1712.00726) [cs.CV]. URL : <https://arxiv.org/abs/1712.00726>.
- [13] Mark EVERINGHAM et al. « The Pascal Visual Object Classes (VOC) challenge ». In : *International Journal of Computer Vision* 88 (juin 2010), p. 303-338. DOI : [10.1007/s11263-009-0275-4](https://doi.org/10.1007/s11263-009-0275-4).

- [14] Caroline A SCHNEIDER, Wayne S RASBAND et Kevin W ELICEIRI. « NIH Image to ImageJ : 25 years of image analysis ». In : *Nature methods* 9.7 (2012), p. 671-675.
- [15] Julian F MILLER, Peter THOMSON, Terence FOGARTY et al. « Designing electronic circuits using evolutionary algorithms. arithmetic circuits : A case study ». In : *Genetic algorithms and evolution strategies in engineering and computer science* 10 (1997).
- [16] M-J WILLIS et al. « Genetic programming : An introduction and survey of applications ». In : *Second international conference on genetic algorithms in engineering systems : innovations and applications*. IET. 1997, p. 314-319.
- [17] Johannes TEXTOR. *Dagitty*. <https://github.com/jtextor/dagitty>. 2010.
- [18] OpenCV TEAM. *OpenCV : Open source Computer Vision Library*. <https://opencv.org/>.
- [19] OpenCV TEAM. *OpenCV : Open source Computer Vision Library - Python*. <https://github.com/opencv/opencv-python>.
- [20] Carsen STRINGER et al. « Cellpose : a generalist algorithm for cellular segmentation ». In : *Nature methods* 18.1 (2021), p. 100-106.
- [21] Marius PACHITARIU et Carsen STRINGER. « Cellpose 2.0 : how to train your own model ». In : *Nature methods* 19.12 (2022), p. 1634-1641.
- [22] Carsen STRINGER et Marius PACHITARIU. « Cellpose3 : one-click image restoration for improved cellular segmentation ». In : *Nature Methods* (2025), p. 1-8.
- [23] Kaiming HE et al. « Mask r-cnn ». In : *Proceedings of the IEEE international conference on computer vision*. 2017, p. 2961-2969.
- [24] Uwe SCHMIDT et al. « Cell detection with star-convex polygons ». In : *Medical image computing and computer assisted intervention—MICCAI 2018 : 21st international conference, Granada, Spain, September 16-20, 2018, proceedings, part II* 11. Springer. 2018, p. 265-273.
- [25] Alexander KIRILLOV et al. « Segment Anything ». In : (2023). arXiv : [2304.02643](https://arxiv.org/abs/2304.02643) [cs.CV]. URL : <https://arxiv.org/abs/2304.02643>.
- [26] Meta FAIR. *SA-V Dataset*. <https://ai.meta.com/datasets/segment-anything-video/>. 2024.
- [27] Rishi BOMMASANI et al. *On the Opportunities and Risks of Foundation Models*. 2022. arXiv : [2108.07258](https://arxiv.org/abs/2108.07258) [cs.LG]. URL : <https://arxiv.org/abs/2108.07258>.
- [28] Ashish VASWANI et al. *Attention Is All You Need*. 2023. arXiv : [1706.03762](https://arxiv.org/abs/1706.03762) [cs.CL]. URL : <https://arxiv.org/abs/1706.03762>.
- [29] Chaitanya RYALI et al. *Hiera : A Hierarchical Vision Transformer without the Bells-and-Whistles*. 2023. arXiv : [2306.00989](https://arxiv.org/abs/2306.00989) [cs.CV]. URL : <https://arxiv.org/abs/2306.00989>.
- [30] Nikhila RAVI et al. « SAM 2 : Segment Anything in Images and Videos ». In : *arXiv preprint arXiv :2408.00714* (2024). URL : <https://arxiv.org/abs/2408.00714>.
- [31] Johannes SCHINDELIN et al. « Fiji : an open-source platform for biological-image analysis ». In : *Nature Methods* 9.7 (2012), p. 676-682. DOI : [10.1038/nmeth.2019](https://doi.org/10.1038/nmeth.2019).

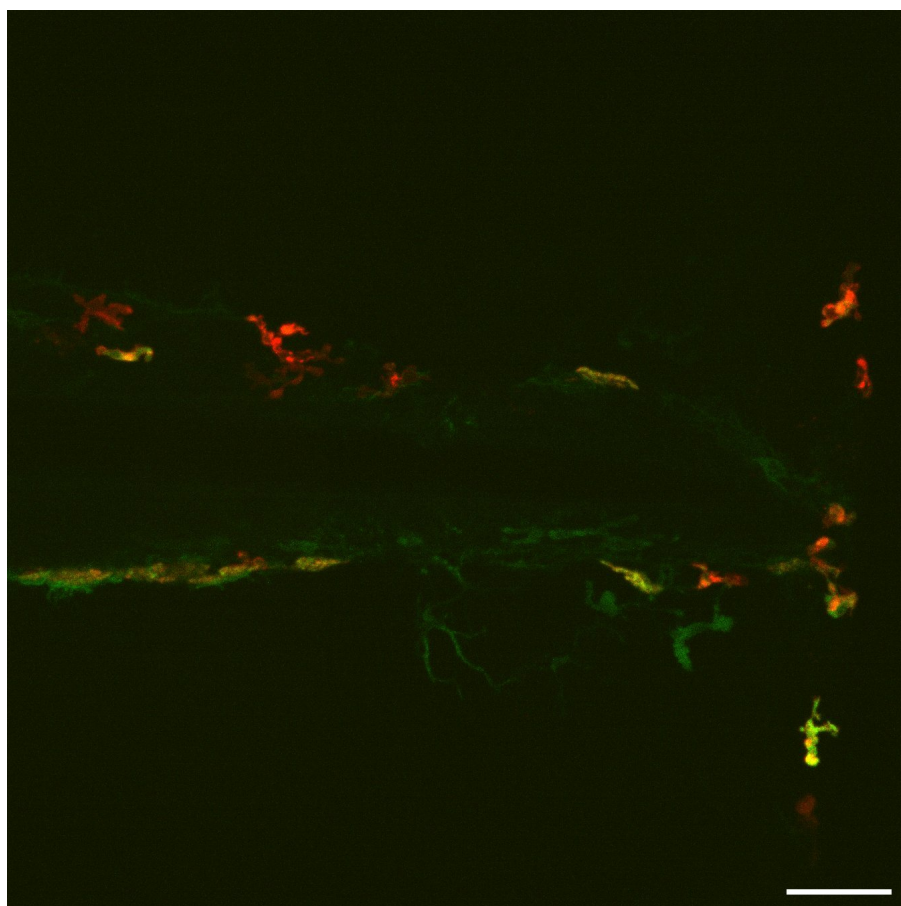
- 
- [32] Ilya LOSHCHILOV et Frank HUTTER. « Decoupled Weight Decay Regularization ». In : (2019). arXiv : [1711.05101 \[cs.LG\]](https://arxiv.org/abs/1711.05101). URL : <https://arxiv.org/abs/1711.05101>.
  - [33] Adam PASZKE et al. « PyTorch : An Imperative Style, High-Performance Deep Learning Library ». In : (déc. 2019). DOI : [10.48550/arXiv.1912.01703](https://doi.org/10.48550/arXiv.1912.01703).



# Annexe A

## Annexes

### A.1 Superposition des deux canaux



**Figure A.1.** Superposition de deux canaux de couleurs, macrophages en rouge et calcium en vert.

## A.2 Liste des fonctions de traitement d'images disponibles dans Kartezio

| Numéro | Fonction         | Description                                                           |
|--------|------------------|-----------------------------------------------------------------------|
| 1      | max              | Maximum                                                               |
| 2      | min              | Minimum                                                               |
| 3      | mean             | Moyenne                                                               |
| 4      | add              | Addition d'images                                                     |
| 5      | subtract         | Soustraction                                                          |
| 6      | bitwise_not      | Opérateur binaire NON                                                 |
| 7      | bitwise_or       | Opérateur binaire OU                                                  |
| 8      | bitwise_and      | Opérateur binaire ET                                                  |
| 9      | bitwise_and_mask | Opérateur binaire d'addition de masque                                |
| 10     | bitwise_xor      | Opérateur binaire OU exclusif                                         |
| 11     | sqrt             | Racine carrée                                                         |
| 12     | pow2             | Puissance carrée                                                      |
| 13     | exp              | Exponentielle                                                         |
| 14     | log              | Logarithme                                                            |
| 15     | median_blur      | Filtre de flou médian                                                 |
| 16     | gaussian_blur    | Filtre de flou gaussien                                               |
| 17     | laplacian        | Calcule le laplacien d'une image                                      |
| 18     | sobel            | Calcule la première, seconde, troisième dérivée d'une image           |
| 19     | robert_cross     | Filtre de Robert Cross                                                |
| 20     | canny            | Trouve les contours dans une image en utilisant l'algorithme de Canny |
| 21     | sharpen          | Filtre d'accentuation des bords                                       |
| 22     | gabor            | Filtre de Gabor                                                       |
| 23     | abs_diff         | Différence absolue terme à terme entre deux matrices                  |
| 24     | abs_diff2        | Différence absolue carrée terme à terme entre deux matrices           |
| 25     | fluo_tophat      | Filtrage morphologique de type Top Hat                                |
| 26     | rel_diff         | Différence relative                                                   |
| 27     | erode            | Erode une image                                                       |
| 28     | dilate           | Dilate une image                                                      |
| 29     | open             | Ouvre une image                                                       |
| 30     | close            | Ferme une image                                                       |
| 31     | morph_gradient   | Gradient morphologique                                                |
| 32     | morph_tophat     | Top Hat morphologique (extrait les objets clairs sur fond noir)       |

| Numéro | Fonction                                   | Description                                                                                    |
|--------|--------------------------------------------|------------------------------------------------------------------------------------------------|
| 33     | <code>morph_blackhat</code>                | Met en évidence les petites zones sombres ou les trous sur fond clair                          |
| 34     | <code>fill_holes</code>                    | Remplit les trous dans les masques                                                             |
| 35     | <code>remove_small_objects</code>          | Retire les petits objets                                                                       |
| 36     | <code>remove_small_holes</code>            | Retire les petits trous dans les masques                                                       |
| 37     | <code>threshold</code>                     | Seuil                                                                                          |
| 38     | <code>threshold_at_1</code>                | Seuil égal à 1 (omet les pixels noirs)                                                         |
| 39     | <code>distance_transform</code>            | Calcule la distance au pixel valant 0 le plus proche pour chaque pixel de l'image              |
| 40     | <code>distance_transform_and_thresh</code> | Calcule la distance au pixel valant le seuil donné le plus proche pour chaque pixel de l'image |
| 41     | <code>inrange_bin</code>                   | Crée un masque binaire                                                                         |
| 42     | <code>inrange</code>                       | Crée un masque pour les pixels différents de 0                                                 |

Table A.1. Fonctions et descriptions

### A.3 Tracking des macrophages par MacTrack

Il s'agit de la version détaillée de la partie 3.1

#### Segmentation d'une vidéo.

La segmentation de la vidéo peut donc enfin commencer après toute cette étape structurelle fastidieuse, mais nécessaire. La première étape de segmentation est réalisée par la fonction `locate` que nous avons présentée dans la partie 2.3. Nous l'appliquons à chacune des images de la vidéo à segmenter. Nous obtenons donc une première segmentation image par image de la vidéo.

Il en sort à la fois les images segmentées, mais aussi des sous-dossiers contenant les segmentations de chaque macrophage détecté image par image. Autrement dit, nous obtenons un dossier contenant 266 images en noir et blanc de la segmentation des macrophages, mais aussi un dossier contenant des sous-dossiers par macrophage qui contiennent chacun les images de la segmentation de ce macrophage précis pour le nombre d'images dans lesquelles il a été repéré. La Fig. 2.10 nous montre un exemple des deux sorties obtenues à l'issue de cette première segmentation.

La première étape de segmentation est donc terminée. Puisque nous faisons de la segmentation sémantique, les macrophages proches peuvent être segmentés comme

un seul objet. De plus, puisqu'on fait de la microscopie confocale en prenant plusieurs images à différentes profondeurs (*z-stacks*), il est possible qu'un macrophage se cache derrière un autre ; on parle d'occlusion dans ce cas.

Il faut donc traiter ces différents cas de figure afin de ne pas fausser le tracking des macrophages. Nous avons donc mis en place un post-traitement permettant de corriger la segmentation obtenue.

### Correction de la segmentation.

Les macrophages sont des cellules qui ont tendance à se regrouper entre eux pour communiquer, on parle de chimio-tactisme. Ce phénomène est particulièrement problématique dans notre cas. En effet, la segmentation que l'on effectue sur les images peut parfois ne pas reconnaître deux objets distincts s'ils sont très proches l'un de l'autre et les experts du domaine ont parfois du mal aussi à les distinguer.

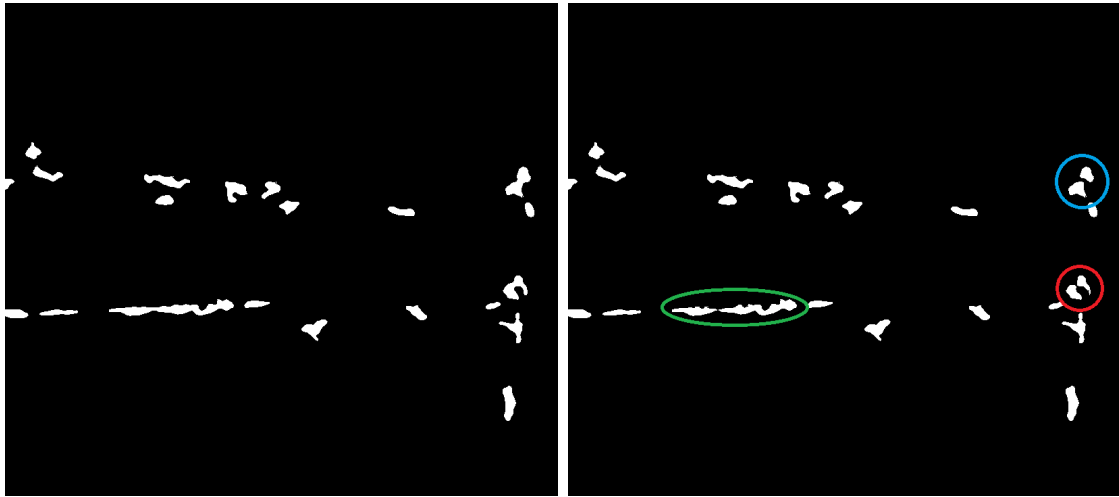
Ainsi, nous allons utiliser l'avantage d'avoir des vidéos et observer ce qu'il se passe dans les images précédant un rapprochement qui résulte en une segmentation unique de deux macrophages différents, à l'aide d'une fonction nommée `defuse`.

Pour  $i$  allant de 2 à  $n$  avec  $n$  étant le nombre d'images dans la vidéo, nous allons regarder l'image  $i$  et l'image  $i - 1$ . Nous calculons ensuite l'IoU entre un objet  $k$  de l'image  $i$  et tous les macrophages de l'image  $i - 1$ . Nous répétons ces calculs pour chacun des macrophages de l'image  $i$ .

Si l'IoU entre le macrophage  $k$  de l'image  $i$  et plus d'un macrophage de l'image  $i - 1$  est strictement supérieure à 0, alors on en déduit que le macrophage  $k$  de l'image  $i$  est en réalité la fusion des différents macrophages. Nous cherchons donc à corriger cette fusion en séparant le macrophage incorrectement fusionné en plusieurs macrophages, chacun correspondant à un des macrophages de l'image  $i - 1$  à l'origine de la fusion.

Pour cela, nous récupérons la segmentation des objets de l'image  $i - 1$  à l'origine de la fusion et nous extrayons la séparation entre ceux-ci. Autrement dit, nous nous retrouvons avec des coordonnées de pixels noirs qui séparent les différents macrophages. Puis, nous reportons cette séparation sur le macrophage  $k$  de l'image  $i$ , résultat de la fusion. Nous forçons donc la séparation de ce macrophage qui a été incorrectement fusionné. La Fig. A.2 nous montre un exemple de ce post-traitement. Cette fonction est aussi appliquée dans le sens contraire de la vidéo, c'est-à-dire de l'image  $n$  à l'image  $n - 1$ . Ainsi, si un macrophage de l'image  $i$  est fusionné avec un macrophage de l'image  $i + 1$ , nous allons le séparer en deux macrophages distincts. Nous avons donc à la fois une correction de la fusion dans le sens dit « avant » mais aussi une correction de la séparation entre plusieurs macrophages dans le sens « arrière » de la vidéo.

L'implémentation d'une telle fonction nous a permis de corriger drastiquement les problèmes d'occlusion et de rapprochement des macrophages, ce qui pouvait résulter en une mauvaise segmentation. En effet, nous avons pu observer qu'après l'application de celle-ci, le nombre de macrophages détectés par image était plus cohérent avec la réalité que nous observons à l'œil nu.



**Figure A.2.** Exemple d'application de la fonction `defuse`.

Cependant, cette implémentation éveille certains problèmes. En effet, lorsque l'on corrige la fusion ou la séparation d'un macrophage, nous perdons en précision, les contours des macrophages sont érodés à cause des seuils utilisés. De plus, appliquer une fonction qui traite à la fois l'image présente mais aussi celle d'avant et d'après, pour toutes les images d'une vidéo, augmente drastiquement le temps de calcul. Plus la vidéo sera longue et plus le temps de calcul sera important. Il est important de prendre ces paramètres en compte.

### Tracking des macrophages.

Nous avons donc maintenant une segmentation corrigée pour chacune des images de la vidéos. Nous avons aussi une liste de macrophages pour chaque image, chacun étant représenté par un masque binaire. Nous cherchons donc à suivre chacun de ces macrophages à travers le temps. Pour cela, nous avons implémenté une fonction nommée `track`.

Nous réalisons une analyse par paire d'images comme pour la correction de la segmentation. En effet, nous examinons chaque image  $i$  avec  $i$  allant de 2 à  $n$ , afin de la comparer avec l'image  $i - 1$ . Pour effectuer cette comparaison, nous allons calculer l'IoU entre un macrophage de l'image  $i$  et chaque macrophage de l'image  $i - 1$ .

Si un macrophage de l'image  $i$  présente une IoU supérieure à un seuil choisi égal à 0.5 avec un macrophage de l'image  $i - 1$ , alors la fonction `track` désigne ces deux macrophages comme étant le même. On leur attribue alors le même identifiant. Nous réalisons cette étape pour tous les macrophages de l'image  $i$  et ce pour toutes les images de la vidéo.

Si un macrophage de l'image  $i$  n'a pas d'IoU supérieure à 0.5 avec n'importe quel macrophage de l'image  $i - 1$ , alors il est considéré comme un nouveau macrophage. On lui attribue donc un nouvel identifiant. À l'inverse, si un macrophage de l'image  $i - 1$  n'a pas d'IoU supérieure à 0.5 avec aucun macrophage de l'image  $i$ , alors il est considéré comme un macrophage perdu. Le tracking de ce macrophage s'arrête donc à l'image  $i - 1$ .

Nous avons appliqué la correction de la segmentation juste avant le tracking des

macrophages. Nous ne sommes donc pas censés obtenir une IoU supérieure à 0.5 entre un macrophage de l'image  $i$  et n'importe quel macrophage de l'image  $i - 1$  si ce n'est pas le même macrophage. En effet, la correction de la segmentation a pour but de séparer les macrophages qui étaient fusionnés dans l'image  $i$  et de les corriger en fonction des macrophages de l'image  $i - 1$ . Ainsi, si un macrophage de l'image  $i$  est fusionné avec un macrophage de l'image  $i - 1$ , alors il sera séparé et on ne pourra pas obtenir une IoU supérieure à 0.5 entre ces deux macrophages. Le tracking est donc très précis. Nous n'avons pas rencontré de cas particuliers où le tracking n'était pas correct.

### Choix du seuil d'IoU.

Nous avons choisi un seuil d'IoU de 0.5 pour le tracking des macrophages pour plusieurs raisons. Tout d'abord, avoir un seuil tel qu'on l'a choisi nous permet d'avoir un suivi efficace des macrophages à travers le temps. En effet, le tracking suit la correction de la segmentation, ce qui signifie qu'il y a une possibilité que deux macrophages ou plus soient proches les uns des autres. Il se peut donc que, dans l'image suivante, la segmentation d'un macrophage empiète sur la segmentation d'un autre. Ainsi, le choix d'un seuil permet de ne pas identifier deux macrophages comme étant le même si leur IoU est trop faible.

De plus, nous avons vu que les macrophages sont des cellules immunitaires très mobiles. Nous avons aussi vu qu'une IoU de 0.5 entre une prédiction et la vérité, par exemple, est considérée comme une prédiction correcte. Nous faisons donc un compromis entre la qualité du suivi et la mobilité des macrophages.

### Photoblanchiment du microscope et seuil sur la durée de tracking.

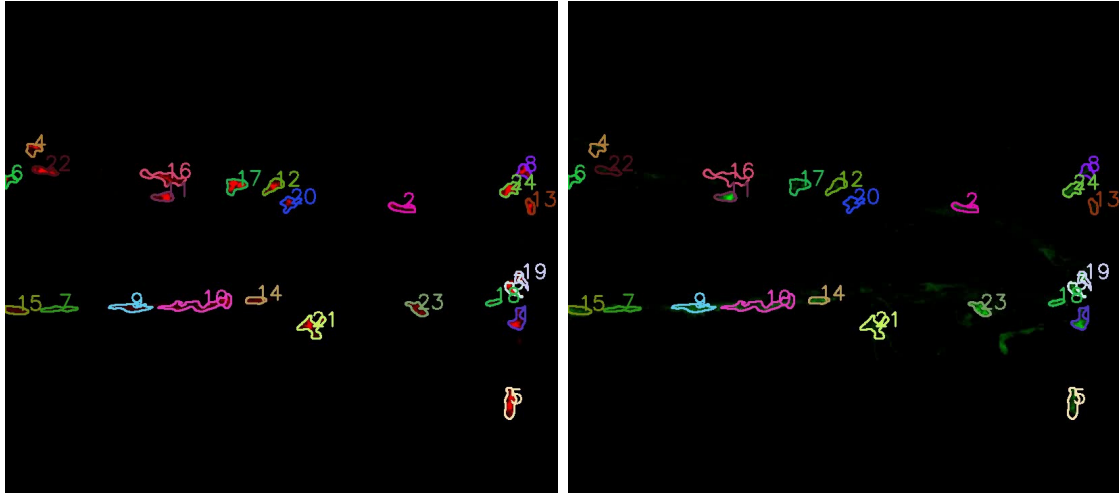
Nous obtenons ainsi un suivi de chaque macrophage à travers le temps. Cependant, en microscopie, le niveau d'intensité exprimé après activation par le laser peut varier d'un poisson à un autre. De plus, plus un poisson est exposé longtemps à la lumière du laser, plus la fluorescence de la protéine observée diminue ; on parle de photoblanchiment (*photobleaching*).

Il est donc possible que certains macrophages soient détectés pendant quelques images puis disparaissent. Nous avons donc mis en place un seuil sur la durée de tracking d'un macrophage. Si un macrophage est détecté pendant moins de 10 images consécutives, alors il est retiré arbitrairement de la liste des macrophages suivis. Cela permet de ne pas avoir de détection d'objets pendant une ou deux images, qui ne sont pas des macrophages dans la plupart des cas.

Nous avons choisi ce seuil de 10 images, ce qui correspond à environ 1 minute 30 secondes d'observation au microscope (une image toutes les 9 secondes), après plusieurs essais. C'est avec ce seuil que nous réduisons le nombre de macrophage détectés à un montant qui nous rapproche du consensus des experts du domaine sur le nombre de macrophages présents au niveau de la nageoire caudale dans les différentes conditions expérimentales (nageoire coupée ou non).

### Résultats et visualisation du tracking des macrophages

Pour visualiser les résultats de la segmentation des vidéos de macrophages, nous avons mis en place la fonction `result`. Celle-ci permet d'ajouter une couche de numérotation et de détourage ce chacun des macrophages détectés dans chaque image de la vidéo d'origine. Nous reportons aussi ce suivi de macrophages dans la vidéo du canal vert. La Fig. A.3 nous montre un exemple de cette visualisation des résultats sur les deux canaux de la vidéo.



**Figure A.3.** Résultats du tracking des macrophages sur une image de chaque canal de la vidéo.

À gauche, nous avons le résultat du tracking sur le canal rouge de la vidéo. À droite, nous avons le résultat du tracking sur le canal vert de la vidéo. Les macrophages sont numérotés et les contours sont dessinés avec une couleur unique par macrophage. Les images proviennent de la vidéo d'un poisson dont la nageoire caudale a été amputée et observée à 3h post-aputuation. Il s'agit de la première image du film.

Des exemples de résultats sous la forme de vidéos sont présentés dans la section **Liens utiles**. Vous trouverez notamment les résultats de la segmentation avant et après le post-traitement.